

Vector Calculus

CS115 - Math for Computer Science

September 9, 2024

Machine learning is about solving an optimization problem whose variables are the parameters of a given model

Solving optimization problems require **gradient information**

Central to this chapter is the concept of the function, which we often write

$$f : \mathbb{R}^D \mapsto \mathbb{R}$$

$$\mathbf{x} \mapsto f(\mathbf{x})$$

Differentiation of Univariate Functions

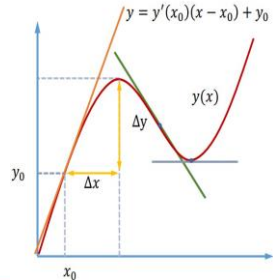
- Difference Quotient.** The average slope of f between x and $x + \partial x$

$$\frac{\partial y}{\partial x} := \frac{f(x + \partial x) - f(x)}{\partial x}$$

- Derivative.** Pointing in the direction of steepest ascent of f .

$$\frac{df}{dx} := \lim_{h \rightarrow 0} \frac{f(x + h) - f(x)}{h}$$

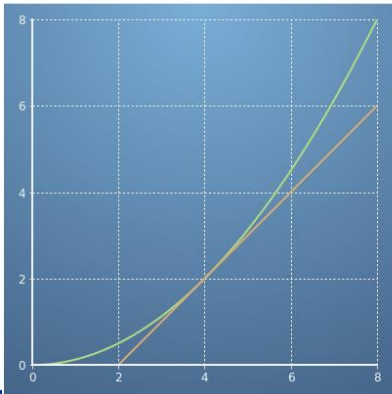
- Unless confusion arises, we often use $f' = \frac{df}{dx}$.



Differentiation of Univariate Functions

Difference Quotient and Derivative

Estimate the gradient of the tangent to the function at the point $(4, 2)$ based on the image below.



The gradient is -1

The gradient is 1

The gradient is 0

The gradient is 2

Taylor Series

- Representation of a function as an infinite sum of terms, using derivatives of evaluated at x_0 .

- **Taylor polynomial.** The Taylor polynomial of degree n of $f : \mathbb{R} \mapsto \mathbb{R}$ at x_0 is:

$$T_n(x) := \sum_{k=0}^n \frac{f^{(k)}(x_0)}{k!} (x - x_0)^k, \text{ where } f^{(k)}(x_0) \text{ is the } k\text{th derivative of } f \text{ at } x_0.$$

- **Taylor Series.** For a smooth function $f \in \mathcal{C}^\infty$, the Taylor series of f at x_0 is:

$$T_\infty(x) := \sum_{k=0}^{\infty} \frac{f^{(k)}(x_0)}{k!} (x - x_0)^k.$$

- If $f(x) = T_\infty(x)$, f is called **analytic**.

Function Approximation: Linearization and More

- First-order approximation of $f(\mathbf{x})$ (i.e., linearization by taking the first two terms of Taylor Series)

$$f(\mathbf{x}) \approx f(\mathbf{x}_0) + (\nabla_{\mathbf{x}} f)(\mathbf{x}_0)(\mathbf{x} - \mathbf{x}_0)$$

- Multivariate Taylor Series for $f : \mathbb{R}^D \mapsto \mathbb{R}$ at $\mathbf{x}_0$

$$f(\mathbf{x}) = \sum_{k=0}^{\infty} \frac{D_{\mathbf{x}}^k f(\mathbf{x}_0)}{k!} \delta^k,$$

where $D_{\mathbf{x}}^k f(\mathbf{x}_0)$ is the k th derivative of f w.r.t. $\mathbf{x}$, evaluated at $\mathbf{x}_0$, and $\delta := \mathbf{x} - \mathbf{x}_0$.

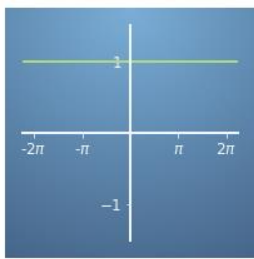
- Partial sum up to, say n , can be an approximation of $f(\mathbf{x})$.
- $D_{\mathbf{x}}^k f(\mathbf{x}_0)$ and δ^k are k th order tensors, i.e., k -dimensional array.
- δ^k is a k -fold outer product $\otimes$. For example, $\delta^2 = \delta \otimes \delta = \delta \delta^T$. $\delta^3 = \delta \otimes \delta \otimes \delta$.

Taylor Series

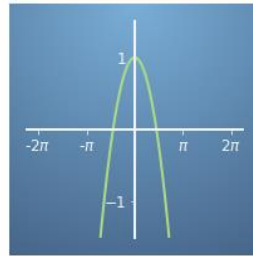
$$e^x = 1 + \frac{x}{1!} + \frac{x^2}{2!} + \frac{x^3}{3!} + \dots + \frac{x^n}{n!} + o(x^n)$$

$$\sin x = x - \frac{x^3}{3!} + \frac{x^5}{5!} - \dots + \frac{(-1)^n}{(2n+1)!} x^{2n+1} + o(x^{2n+1})$$

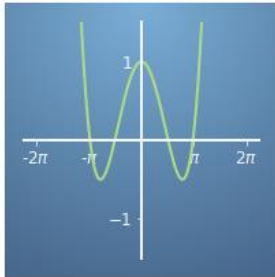
$$\cos x = 1 - \frac{x^2}{2!} + \frac{x^4}{4!} - \dots + \frac{(-1)^n}{(2n)!} x^{2n} + o(x^{2n}).$$



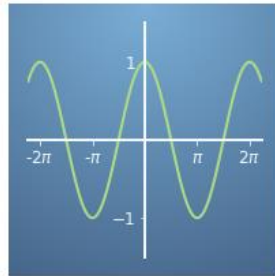
$$f_0(x) = 1$$



$$f_2(x) = 1 - \frac{x^2}{2}$$



$$f_4(x) = 1 - \frac{x^2}{2} + \frac{x^4}{24}$$



$$f(x) = \cos(x)$$

Partial Differentiation and Gradients

- Now, $f : \mathbb{R}^n \mapsto \mathbb{R}$.
- Gradient of f w.r.t. $\mathbf{x}$ $\nabla_{\mathbf{x}} f$: Varying one variable at a time and keeping the others constant.

Partial Derivative. For $f : \mathbb{R}^n \mapsto \mathbb{R}$,

$$\frac{\partial f}{\partial x_1} = \lim_{h \rightarrow 0} \frac{f(x_1 + h, x_2, \dots, x_n) - f(\mathbf{x})}{h}$$

$\vdots$

$$\frac{\partial f}{\partial x_n} = \lim_{h \rightarrow 0} \frac{f(x_1, x_2, \dots, x_n + h) - f(\mathbf{x})}{h}$$

Gradient. Get the partial derivatives and collect them in the row vector.

$$\nabla_{\mathbf{x}} f = \frac{df}{d\mathbf{x}} = \left(\frac{\partial f(\mathbf{x})}{\partial x_1} \quad \dots \quad \frac{\partial f(\mathbf{x})}{\partial x_n} \right) \in \mathbb{R}^{1 \times n}$$

Rules for Partial Differentiation

- Product rule

$$\frac{\partial}{\partial \mathbf{x}} (f(\mathbf{x})g(\mathbf{x})) = \frac{\partial f}{\partial \mathbf{x}} g(\mathbf{x}) + f(\mathbf{x}) \frac{\partial g}{\partial \mathbf{x}}$$

- Sum rule

$$\frac{\partial}{\partial \mathbf{x}} (f(\mathbf{x}) + g(\mathbf{x})) = \frac{\partial f}{\partial \mathbf{x}} + \frac{\partial g}{\partial \mathbf{x}}$$

- Chain rule

$$\frac{\partial}{\partial \mathbf{x}} g(f(\mathbf{x})) = \frac{\partial g}{\partial f} \frac{\partial f}{\partial \mathbf{x}}$$

Example

$$f(x, y) = (x + 2y^3)^2$$

$$\frac{\partial f(x, y)}{\partial x}$$

$$\frac{\partial f(x, y)}{\partial y}$$

$$\frac{\partial f(x, y)}{\partial x} = 2(x + 2y^3) \frac{\partial x + 2y^3}{\partial x} = 2(x + 2y^3)$$

$$\frac{\partial f(x, y)}{\partial y} = 2(x + 2y^3) \frac{\partial x + 2y^3}{\partial y} = 12(x + 2y^3)y^2$$

Example

$$f(x_1, x_2) = x_1^2 x_2 + x_1 x_2^3$$

$$\nabla_{(x_1, x_2)} f$$

$$\nabla_{(x_1, x_2)} f = \frac{df}{dx} = \left(\frac{\partial f(x_1, x_2)}{\partial x_1} \quad \frac{\partial f(x_1, x_2)}{\partial x_2} \right) = (2x_1 x_2 + x_2^3 \quad x_1^2 + 3x_1 x_2^2)$$

Chain Rule

- $f : \mathbb{R}^2 \mapsto \mathbb{R}$ of two variables x_1 and x_2 . $x_1(t)$ and $x_2(t)$ are functions of t .

$$\frac{df}{dt}$$

$$\frac{df}{dt} = \left(\frac{\partial f}{\partial x_1} \quad \frac{\partial f}{\partial x_2} \right) \begin{pmatrix} \frac{\partial x_1(t)}{\partial t} \\ \frac{\partial x_2(t)}{\partial t} \end{pmatrix} = \frac{\partial f}{\partial x_1} \frac{\partial x_1}{\partial t} + \frac{\partial f}{\partial x_2} \frac{\partial x_2}{\partial t}$$

Chain Rule

- Example.** $f(x_1, x_2) = x_1^2 + 2x_2$, where $x_1(t) = \sin(t)$, $x_2(t) = \cos(t)$

$$\frac{df}{dt} = \frac{\partial f}{\partial x_1} \frac{\partial x_1}{\partial t} + \frac{\partial f}{\partial x_2} \frac{\partial x_2}{\partial t} = 2 \sin(t) \cos(t) - 2 \sin t = 2 \sin(t)(\cos(t) - 1)$$

Chain Rule

- $f : \mathbb{R}^2 \mapsto \mathbb{R}$ of two variables x_1 and x_2 . $x_1(s, t)$ and $x_2(s, t)$ are functions of s, t .

$$\frac{\partial f}{\partial s} = \frac{\partial f}{\partial x_1} \frac{\partial x_1}{\partial s} + \frac{\partial f}{\partial x_2} \frac{\partial x_2}{\partial s}$$

$$\frac{\partial f}{\partial t} = \frac{\partial f}{\partial x_1} \frac{\partial x_1}{\partial t} + \frac{\partial f}{\partial x_2} \frac{\partial x_2}{\partial t}$$

$$\begin{aligned} \frac{df}{d(s, t)} &= \frac{\partial f}{\partial \mathbf{x}} \frac{\partial \mathbf{x}}{\partial (s, t)} = \underbrace{\begin{bmatrix} \frac{\partial f}{\partial x_1} & \frac{\partial f}{\partial x_2} \end{bmatrix}}_{\frac{\partial f}{\partial \mathbf{x}}} \underbrace{\begin{bmatrix} \frac{\partial x_1}{\partial s} & \frac{\partial x_1}{\partial t} \\ \frac{\partial x_2}{\partial s} & \frac{\partial x_2}{\partial t} \end{bmatrix}}_{\frac{\partial \mathbf{x}}{\partial (s, t)}} \\ &= \frac{\partial f}{\partial \mathbf{x}} \frac{\partial \mathbf{x}}{\partial (s, t)} \end{aligned}$$

Example

Given that $f(x, y) = \pi x^2 y$, $x(t) = t^2 + 1$, and $y(t) = t^2 - 1$, calculate the total derivative $\frac{df}{dt}$.

$$\frac{df}{dt} = 4\pi t(t^2 + 1)(t^2 - 1) + 2\pi t(t^2 + 1)^2$$

Example

$$f(\mathbf{x}) = f(x_1, x_2) = x_1^2 - x_2^2$$

$$x_1(u_1, u_2) = 2u_1 + 3u_2$$

$$x_2(u_1, u_2) = 2u_1 - 3u_2$$

$$u_1(t) = \cos(t/2)$$

$$u_2(t) = \sin(2t)$$

$$\frac{df}{dt}$$

Example

$$f(\mathbf{x}) = f(x_1, x_2) = x_1^2 - x_2^2$$

$$x_1(u_1, u_2) = 2u_1 + 3u_2$$

$$x_2(u_1, u_2) = 2u_1 - 3u_2$$

$$u_1(t) = \cos(t/2)$$

$$u_2(t) = \sin(2t)$$

$$\frac{df}{dt} = \frac{\partial f}{\partial \mathbf{x}} \frac{\partial \mathbf{x}}{\partial \mathbf{u}} \frac{d\mathbf{u}}{dt} = [2x_1, -2x_2] \begin{bmatrix} 2 & 3 \\ 2 & -3 \end{bmatrix} \begin{bmatrix} -\sin(t/2)/2 \\ 2\cos(2t) \end{bmatrix}$$

Compute $\frac{df}{dx}$ of the following function using the chain rule. ogy

$$a = x^2$$

$$b = \exp(a)$$

$$c = a + b$$

$$d = \log(c)$$

$$e = \sin(c)$$

$$f = d + e$$

$$\frac{df}{dx} = \frac{(1 + \cos(x^2 + \exp(x^2)))(x^2 + \exp(x^2))(2x + 2x \exp(x^2))}{x^2 + \exp(x^2)}$$

Gradients of Vector-Valued Function

- For a function $\mathbf{f} : \mathbb{R}^n \mapsto \mathbb{R}^m$ and vector $\mathbf{x} = (x_1 \dots x_n)^T \in \mathbb{R}^n$, the vector-valued function is:

$$\mathbf{f}(\mathbf{x}) = \begin{pmatrix} f_1(\mathbf{x}) \\ \vdots \\ f_m(\mathbf{x}) \end{pmatrix}$$

- Partial derivative w.r.t. x_i is a column vector: $\frac{\partial \mathbf{f}}{\partial x_i} = \begin{pmatrix} \frac{\partial f_1}{\partial x_i} \\ \vdots \\ \frac{\partial f_m}{\partial x_i} \end{pmatrix}$
- Gradient (or Jacobian): $\frac{d\mathbf{f}(\mathbf{x})}{d\mathbf{x}} = \left(\frac{\partial \mathbf{f}(\mathbf{x})}{\partial x_1} \dots \frac{\partial \mathbf{f}(\mathbf{x})}{\partial x_n} \right)$

Jacobian

$$\begin{aligned} \mathbf{J} &= \nabla_{\mathbf{x}} \mathbf{f} = \frac{d\mathbf{f}(\mathbf{x})}{d\mathbf{x}} = \left(\frac{\partial \mathbf{f}(\mathbf{x})}{\partial x_1} \quad \dots \quad \frac{\partial \mathbf{f}(\mathbf{x})}{\partial x_n} \right) \\ &= \begin{pmatrix} \frac{\partial f_1(\mathbf{x})}{\partial x_1} & \dots & \frac{\partial f_1(\mathbf{x})}{\partial x_n} \\ \vdots & & \vdots \\ \frac{\partial f_m(\mathbf{x})}{\partial x_1} & \dots & \frac{\partial f_m(\mathbf{x})}{\partial x_n} \end{pmatrix} \end{aligned}$$

- For a $\mathbb{R}^n \mapsto \mathbb{R}^m$ function, its Jacobian is a $m \times n$ matrix.

Jacobian

For the function $u(x, y, z) = 9x^2y^2 + ze^x$, $v(x, y, z) = xy + x^2y^3 + 2z$ and $w(x, y, z) = \cos(x)\sin(z)e^y$, calculate the Jacobian matrix and evaluate at the point $(0, 0, 0)$.

Gradients of Vector-Valued Functions

$$\mathbf{f}(\mathbf{x}) = \mathbf{A}\mathbf{x}, \mathbf{f} : \mathbb{R}^n \mapsto \mathbb{R}^m, \mathbf{A} \in \mathbb{R}^{m \times n}, \mathbf{x} \in \mathbb{R}^n$$

$$f_i(\mathbf{x}) = \sum_{j=1}^n A_{ij}x_j$$

$$\frac{\partial f_i}{\partial x_j} = A_{ij}$$

$$\frac{d\mathbf{f}}{d\mathbf{x}} = \begin{pmatrix} \frac{\partial f_1}{\partial x_1} & \cdots & \frac{\partial f_1}{\partial x_n} \\ \vdots & & \vdots \\ \frac{\partial f_m}{\partial x_1} & \cdots & \frac{\partial f_m}{\partial x_n} \end{pmatrix} = \begin{pmatrix} A_{11} & \cdots & A_{1n} \\ \vdots & & \vdots \\ A_{m1} & \cdots & A_{mn} \end{pmatrix} = \mathbf{A}$$

Gradient of Vectors for Matrices

Ví dụ $f(\mathbf{x}) = \mathbf{Ax}$, $\mathbf{f} \in \mathbb{R}^m$, $\mathbf{A} \in \mathbb{R}^{m \times n}$, $\mathbf{x} \in \mathbb{R}^n$ $\frac{df}{d\mathbf{A}}?$

$$\mathbf{f} : \mathbb{R}^{m \times n} \mapsto \mathbb{R}^m \quad \frac{d\mathbf{f}}{d\mathbf{A}} \in \mathbb{R}^{m \times (m \times n)}$$

$$\frac{\partial f_i}{\partial \mathbf{A}} \in \mathbb{R}^{1 \times (m \times n)} \quad \frac{d\mathbf{f}}{d\mathbf{A}} = \begin{pmatrix} \frac{\partial f_1}{\partial \mathbf{A}} \\ \vdots \\ \frac{\partial f_m}{\partial \mathbf{A}} \end{pmatrix}$$

Gradient of Vectors for Matrices

$$\mathbf{f}(\mathbf{x}) = \mathbf{A}\mathbf{x}, \mathbf{f} \in \mathbb{R}^m, \mathbf{A} \in \mathbb{R}^{m \times n}, \mathbf{x} \in \mathbb{R}^n \quad \frac{d\mathbf{f}}{d\mathbf{A}}?$$

$$f_i = \sum_{j=1}^n A_{ij}x_j, \quad i = 1, \dots, m \implies \frac{\partial f_i}{\partial A_{iq}} = x_q$$

$$\frac{\partial f_i}{\partial A_{i.}} = \mathbf{x}^T \in \mathbb{R}^{1 \times 1 \times n}$$

$$\frac{\partial f_i}{\partial A_{k \neq i.}} = 0^T \in \mathbb{R}^{1 \times 1 \times n}, \quad k \neq i$$

$$\frac{\partial f_i}{\partial \mathbf{A}} = \begin{pmatrix} 0^T \\ \vdots \\ 0^T \\ \mathbf{x}^T \\ 0^T \\ \vdots \\ 0^T \end{pmatrix} \in \mathbb{R}^{1 \times (m \times n)}$$

Gradient of Matrices for Matrices (2)

- $R \in \mathbb{R}^{m \times n}$ and $f : \mathbb{R}^{m \times n} \mapsto \mathbb{R}^{n \times n}$ with $f(R) = K := R^T R \in \mathbb{R}^{n \times n}$. What is $\frac{dK}{dR} \in \mathbb{R}^{(n \times n) \times (m \times n)}$?

Gradient of Matrices for Matrices

- $\mathbf{R} \in \mathbb{R}^{m \times n}$ and $\mathbf{f} : \mathbb{R}^{m \times n} \mapsto \mathbb{R}^{n \times n}$ with $\mathbf{f}(\mathbf{R}) = \mathbf{K} := \mathbf{R}^T \mathbf{R} \in \mathbb{R}^{n \times n}$. What is $\frac{d\mathbf{K}}{d\mathbf{R}} \in \mathbb{R}^{(n \times n) \times (m \times n)}$?
- $\frac{dK_{pq}}{d\mathbf{R}} \in \mathbb{R}^{1 \times m \times n}$. Let $\mathbf{r}_i$ be the i th column of $\mathbf{R}$. Then $K_{pq} = \mathbf{r}_p^T \mathbf{r}_q = \sum_{k=1}^m R_{kp} R_{kq}$.
- Partial derivative $\frac{\partial K_{pq}}{\partial R_{ij}}$

$$\frac{\partial K_{pq}}{\partial R_{ij}} = \sum_{k=1}^m \frac{\partial}{\partial R_{ij}} R_{kp} R_{kq} = \partial_{pqij}, \quad \partial_{pqij} = \begin{cases} R_{iq} & \text{if } j = p, p \neq q \\ R_{ip} & \text{if } j = q, p \neq q \\ 2R_{iq} & \text{if } j = p, p = q \\ 0 & \text{otherwise} \end{cases}$$

$$\frac{\partial}{\partial \mathbf{X}} f(\mathbf{X})^\top = \left(\frac{\partial f(\mathbf{X})}{\partial \mathbf{X}} \right)^\top$$

$$\frac{\partial}{\partial \mathbf{X}} \text{tr}(f(\mathbf{X})) = \text{tr} \left(\frac{\partial f(\mathbf{X})}{\partial \mathbf{X}} \right)$$

$$\frac{\partial}{\partial \mathbf{X}} \det(f(\mathbf{X})) = \det(f(\mathbf{X})) \text{tr} \left(f(\mathbf{X})^{-1} \frac{\partial f(\mathbf{X})}{\partial \mathbf{X}} \right)$$

$$\frac{\partial}{\partial \mathbf{X}} f(\mathbf{X})^{-1} = -f(\mathbf{X})^{-1} \frac{\partial f(\mathbf{X})}{\partial \mathbf{X}} f(\mathbf{X})^{-1}$$

$$\frac{\partial \mathbf{a}^\top \mathbf{X}^{-1} \mathbf{b}}{\partial \mathbf{X}} = -(\mathbf{X}^{-1})^\top \mathbf{a} \mathbf{b}^\top (\mathbf{X}^{-1})^\top$$

$$\frac{\partial \mathbf{x}^\top \mathbf{a}}{\partial \mathbf{x}} = \mathbf{a}^\top$$

$$\frac{\partial \mathbf{a}^\top \mathbf{x}}{\partial \mathbf{x}} = \mathbf{a}^\top$$

$$\frac{\partial \mathbf{a}^\top \mathbf{X} \mathbf{b}}{\partial \mathbf{X}} = \mathbf{a} \mathbf{b}^\top$$

$$\frac{\partial \mathbf{x}^\top \mathbf{B} \mathbf{x}}{\partial \mathbf{x}} = \mathbf{x}^\top (\mathbf{B} + \mathbf{B}^\top)$$

$$\frac{\partial}{\partial \mathbf{s}} (\mathbf{x} - \mathbf{A} \mathbf{s})^\top \mathbf{W} (\mathbf{x} - \mathbf{A} \mathbf{s}) = -2(\mathbf{x} - \mathbf{A} \mathbf{s})^\top \mathbf{W} \mathbf{A}$$

Higher-Order Derivatives

- For $f : \mathbb{R}^n \mapsto \mathbb{R}$ of variable $\mathbf{x} \in \mathbb{R}^n$, $\nabla_{\mathbf{x}} f = \frac{df}{d\mathbf{x}} = \left(\frac{\partial f(\mathbf{x})}{\partial x_1} \quad \dots \quad \frac{\partial f(\mathbf{x})}{\partial x_n} \right) \in \mathbb{R}^{1 \times n}$
 - If f is twice-differentiable, the order doesn't matter.

$$H_{\mathbf{x}} f = \begin{pmatrix} \frac{\partial^2 f}{\partial x_1^2} & \frac{\partial^2 f}{\partial x_1 \partial x_2} & \dots & \frac{\partial^2 f}{\partial x_1 \partial x_n} \\ \vdots & & & \vdots \\ \frac{\partial^2 f}{\partial x_1 \partial x_n} & \frac{\partial^2 f}{\partial x_2 \partial x_n} & \dots & \frac{\partial^2 f}{\partial x_n \partial x_n} \end{pmatrix}$$

- For $f : \mathbb{R}^n \mapsto \mathbb{R}^m$, $\nabla_{\mathbf{x}} f \in \mathbb{R}^{m \times n}$
 - Thus, $H_{\mathbf{x}} f \in \mathbb{R}^{m \times n \times n}$ (a tensor)

Higher-Order Derivatives

Calculate the Hessian for the function $f(x, y, z) = xy + \sin(y)\sin(z) + z^3e^x$.

$$H = \begin{bmatrix} e^x z^3 & 1 & 3e^x z^2 \\ 1 & -\sin(y)\sin(z) & \cos(y)\cos(z) \\ 3e^x z^2 & \cos(y)\cos(z) & 6e^x z - \sin(y)\sin(z) \end{bmatrix}$$

Higher-Order Derivatives

Some optimization algorithms (e.g., Newton's method) require second-order derivatives, if they exist.

- (Truncated) Taylor series is often used as an approximation of a function

Backpropagation and Neural Networks

Where we are...

$$s = f(x; W) = Wx$$

scores function

$$L_i = \sum_{j \neq y_i} \max(0, s_j - s_{y_i} + 1)$$

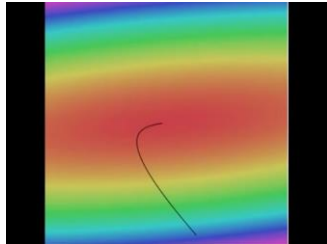
SVM loss

$$L = \frac{1}{N} \sum_{i=1}^N L_i + \sum_k W_k^2$$

data loss + regularization

want $\nabla_W L$

Optimization



```
# Vanilla Gradient Descent
```

```
while True:
    weights_grad = evaluate_gradient(loss_fun, data, weights)
    weights += - step_size * weights_grad # perform parameter update
```

[Landscape image](#) is [CC0 1.0](#) public domain
[Walking man image](#) is [CC0 1.0](#) public domain

Gradient descent

$$\frac{df(x)}{dx} = \lim_{h \rightarrow 0} \frac{f(x+h) - f(x)}{h}$$

Numerical gradient: slow :(), approximate :(), easy to write :)

Analytic gradient: fast :), exact :), error-prone :(

In practice: Derive analytic gradient, check your implementation with numerical gradient

Motivation: Neural Networks with Many Layers

- In a neural network with many layers, the function $\mathbf{y}$ is a many-level function compositions

$$\mathbf{y} = (f_K \circ f_{K-1} \circ \cdots \circ f_1)(\mathbf{x}),$$

where, for example,

- $\mathbf{x}$: images as inputs, $\mathbf{y}$: class labels (e.g., cat or dog) as outputs
 - each f_i has its own parameters
 - In neural networks, with the model parameters $\theta = \{\mathbf{A}_0, \mathbf{b}_0, \dots, \mathbf{A}_{K-1}, \mathbf{b}_{K-1}\}$

$$\left\{ \begin{array}{l} \mathbf{f}_0 := \mathbf{x} \\ \mathbf{f}_1 := \sigma_1(\mathbf{A}_0 \mathbf{f}_0 + \mathbf{b}_0) \\ \vdots \\ \mathbf{f}_K := \sigma_K(\mathbf{A}_{K-1} \mathbf{f}_{K-1} + \mathbf{b}_{K-1}) \end{array} \right.$$

- Minimizing the loss function over θ :
$$\min_{\theta} L(\theta),$$

where $L(\theta) = \|\mathbf{y} - \mathbf{f}_K(\theta, \mathbf{x})\|^2$
- σ_i is called the **activation function** at i -th layer

Motivation: Neural Networks with Many Layers

- In neural networks, with the model parameters $\theta = \{\mathbf{A}_0, \mathbf{b}_0, \dots, \mathbf{A}_{K-1}, \mathbf{b}_{K-1}\}$

$$\begin{cases} \mathbf{f}_0 &:= \mathbf{x} \\ \mathbf{f}_1 &:= \sigma_1(\mathbf{A}_0 \mathbf{f}_0 + \mathbf{b}_0) \\ \vdots & \\ \mathbf{f}_K &:= \sigma_K(\mathbf{A}_{K-1} \mathbf{f}_{K-1} + \mathbf{b}_{K-1}) \end{cases}$$

- σ_i is called the activation function at i -th layer

- Minimizing the loss function over θ :

$$\min_{\theta} L(\theta),$$

$$\text{where } L(\theta) = \|\mathbf{y} - \mathbf{f}_K(\theta, \mathbf{x})\|^2$$

How can we efficiently compute $\frac{dL}{d\theta}$ in computers?

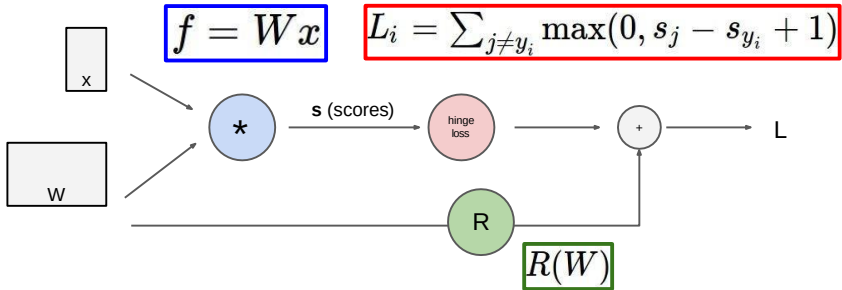
$$\frac{\partial L}{\partial \theta_{K-1}} = \frac{\partial L}{\partial f_K} \frac{\partial f_K}{\partial \theta_{K-1}}$$

$$\frac{\partial L}{\partial \theta_{K-2}} = \frac{\partial L}{\partial f_K} \boxed{\frac{\partial f_K}{\partial f_{K-1}} \frac{\partial f_{K-1}}{\partial \theta_{K-2}}}$$

$$\frac{\partial L}{\partial \theta_{K-3}} = \frac{\partial L}{\partial f_K} \frac{\partial f_K}{\partial f_{K-1}} \boxed{\frac{\partial f_{K-1}}{\partial f_{K-2}} \frac{\partial f_{K-2}}{\partial \theta_{K-3}}}$$

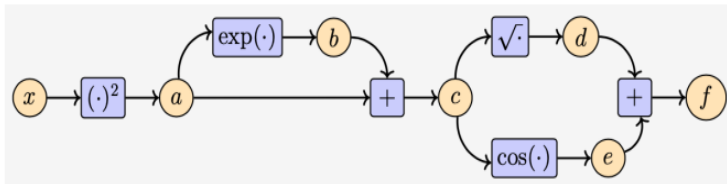
$$\frac{\partial L}{\partial \theta_i} = \frac{\partial L}{\partial f_K} \frac{\partial f_K}{\partial f_{K-1}} \dots \boxed{\frac{\partial f_{i+2}}{\partial f_{i+1}} \frac{\partial f_{i+1}}{\partial \theta_i}}$$

Computational graphs



Backpropagation: Example

- $f(x) = \sqrt{x^2 + \exp(x^2)} + \cos(x^2 + \exp(x^2))$
- Computation graph: Connect via “elementary” operations



$$a = x^2, \quad b = \exp(a), \quad c = a + b, \quad d = \sqrt{c}, \quad e = \cos(c), \quad f = d + e$$

Automatic Differentiation

- A set of techniques to numerically (not symbolically) evaluate the gradient of a function by working with intermediate variables and applying the chain rule.

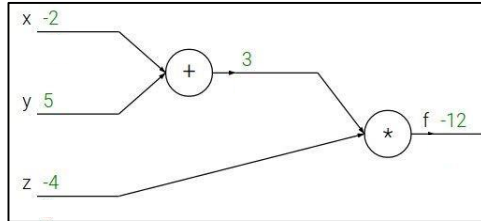
Backpropagation

- Implementation of gradients can be very expensive, unless we are careful.
- Using the idea of automatic differentiation, the whole gradient computation is decomposed into a set of gradients of elementary functions and application of the chain rule.
- Why **backward**?
 - In neural networks, the input dimensionality is often much higher than the dimensionality of labels.
 - In this case, the backward computation (than the forward computation) is much cheaper.
- Works if the target is expressed as a computation graph whose elementary functions are differentiable. If not, some care needs to be taken.

Backpropagation: a simple example

$$f(x, y, z) = (x + y)z$$

e.g. $x = -2$, $y = 5$, $z = -4$



Backpropagation: a simple example

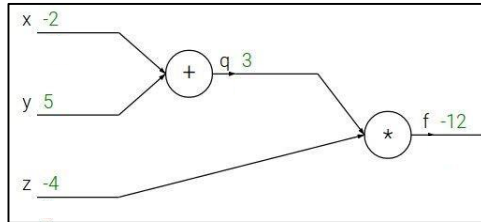
$$f(x, y, z) = (x + y)z$$

e.g. $x = -2, y = 5, z = -4$

$$q = x + y \quad \frac{\partial q}{\partial x} = 1, \frac{\partial q}{\partial y} = 1$$

$$f = qz \quad \frac{\partial f}{\partial q} = z, \frac{\partial f}{\partial z} = q$$

Want: $\frac{\partial f}{\partial x}, \frac{\partial f}{\partial y}, \frac{\partial f}{\partial z}$



Backpropagation: a simple example

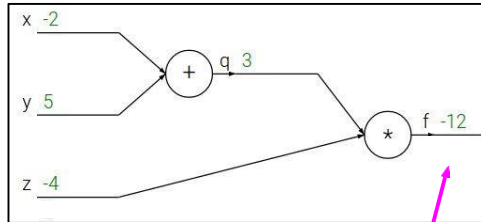
$$f(x, y, z) = (x + y)z$$

e.g. $x = -2, y = 5, z = -4$

$$q = x + y \quad \frac{\partial q}{\partial x} = 1, \frac{\partial q}{\partial y} = 1$$

$$f = qz \quad \frac{\partial f}{\partial q} = z, \frac{\partial f}{\partial z} = q$$

Want: $\frac{\partial f}{\partial x}, \frac{\partial f}{\partial y}, \frac{\partial f}{\partial z}$



$$\frac{\partial f}{\partial f}$$

Backpropagation: a simple example

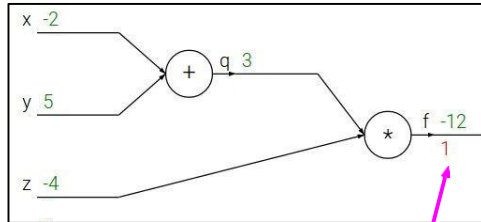
$$f(x, y, z) = (x + y)z$$

e.g. $x = -2, y = 5, z = -4$

$$q = x + y \quad \frac{\partial q}{\partial x} = 1, \frac{\partial q}{\partial y} = 1$$

$$f = qz \quad \frac{\partial f}{\partial q} = z, \frac{\partial f}{\partial z} = q$$

Want: $\frac{\partial f}{\partial x}, \frac{\partial f}{\partial y}, \frac{\partial f}{\partial z}$



$$\frac{\partial f}{\partial f}$$

Backpropagation: a simple example

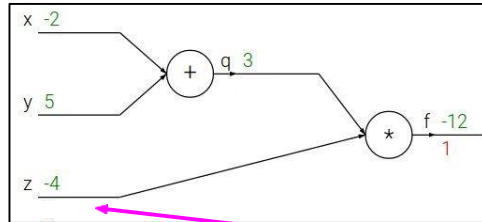
$$f(x, y, z) = (x + y)z$$

e.g. $x = -2, y = 5, z = -4$

$$q = x + y \quad \frac{\partial q}{\partial x} = 1, \frac{\partial q}{\partial y} = 1$$

$$f = qz \quad \frac{\partial f}{\partial q} = z, \frac{\partial f}{\partial z} = q$$

Want: $\frac{\partial f}{\partial x}, \frac{\partial f}{\partial y}, \frac{\partial f}{\partial z}$



$$\frac{\partial f}{\partial z}$$

Backpropagation: a simple example

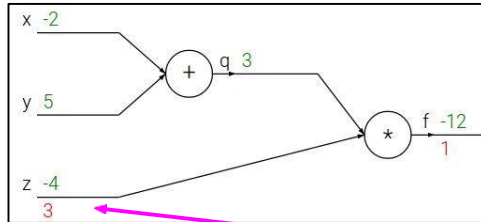
$$f(x, y, z) = (x + y)z$$

e.g. $x = -2, y = 5, z = -4$

$$q = x + y \quad \frac{\partial q}{\partial x} = 1, \frac{\partial q}{\partial y} = 1$$

$$f = qz \quad \frac{\partial f}{\partial q} = z, \frac{\partial f}{\partial z} = q$$

Want: $\frac{\partial f}{\partial x}, \frac{\partial f}{\partial y}, \frac{\partial f}{\partial z}$



$$\frac{\partial f}{\partial z}$$

Backpropagation: a simple example

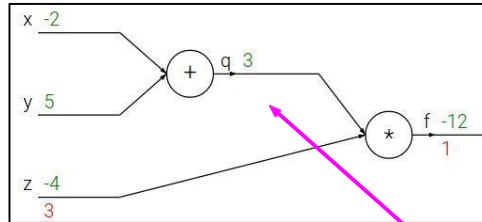
$$f(x, y, z) = (x + y)z$$

e.g. $x = -2, y = 5, z = -4$

$$q = x + y \quad \frac{\partial q}{\partial x} = 1, \frac{\partial q}{\partial y} = 1$$

$$f = qz \quad \frac{\partial f}{\partial q} = z, \frac{\partial f}{\partial z} = q$$

Want: $\frac{\partial f}{\partial x}, \frac{\partial f}{\partial y}, \frac{\partial f}{\partial z}$



$$\frac{\partial f}{\partial q}$$

Backpropagation: a simple example

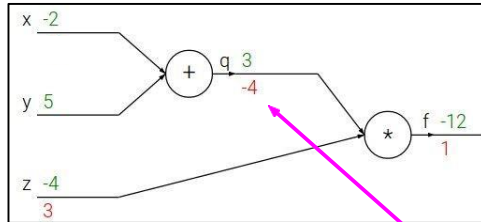
$$f(x, y, z) = (x + y)z$$

e.g. $x = -2, y = 5, z = -4$

$$q = x + y \quad \frac{\partial q}{\partial x} = 1, \frac{\partial q}{\partial y} = 1$$

$$f = qz \quad \frac{\partial f}{\partial q} = z, \frac{\partial f}{\partial z} = q$$

Want: $\frac{\partial f}{\partial x}, \frac{\partial f}{\partial y}, \frac{\partial f}{\partial z}$



$$\frac{\partial f}{\partial q}$$

Backpropagation: a simple example

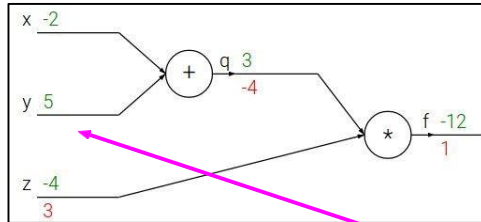
$$f(x, y, z) = (x + y)z$$

e.g. $x = -2, y = 5, z = -4$

$$q = x + y \quad \frac{\partial q}{\partial x} = 1, \frac{\partial q}{\partial y} = 1$$

$$f = qz \quad \frac{\partial f}{\partial q} = z, \frac{\partial f}{\partial z} = q$$

Want: $\frac{\partial f}{\partial x}, \frac{\partial f}{\partial y}, \frac{\partial f}{\partial z}$



$$\frac{\partial f}{\partial y}$$

Backpropagation: a simple example

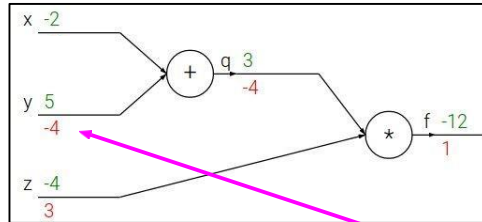
$$f(x, y, z) = (x + y)z$$

e.g. $x = -2, y = 5, z = -4$

$$q = x + y \quad \frac{\partial q}{\partial x} = 1, \frac{\partial q}{\partial y} = 1$$

$$f = qz \quad \frac{\partial f}{\partial q} = z, \frac{\partial f}{\partial z} = q$$

Want: $\frac{\partial f}{\partial x}, \frac{\partial f}{\partial y}, \frac{\partial f}{\partial z}$



$$\frac{\partial f}{\partial y}$$

Chain rule:

$$\frac{\partial f}{\partial y} = \frac{\partial f}{\partial q} \frac{\partial q}{\partial y}$$

Backpropagation: a simple example

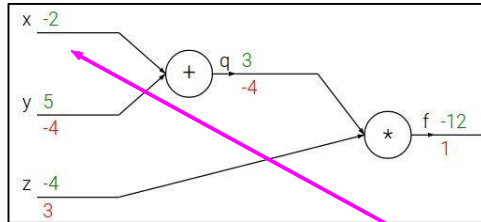
$$f(x, y, z) = (x + y)z$$

e.g. $x = -2, y = 5, z = -4$

$$q = x + y \quad \frac{\partial q}{\partial x} = 1, \frac{\partial q}{\partial y} = 1$$

$$f = qz \quad \frac{\partial f}{\partial q} = z, \frac{\partial f}{\partial z} = q$$

Want: $\frac{\partial f}{\partial x}, \frac{\partial f}{\partial y}, \frac{\partial f}{\partial z}$



$$\frac{\partial f}{\partial x}$$

Backpropagation: a simple example

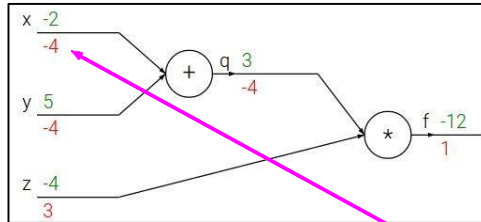
$$f(x, y, z) = (x + y)z$$

e.g. $x = -2, y = 5, z = -4$

$$q = x + y \quad \frac{\partial q}{\partial x} = 1, \frac{\partial q}{\partial y} = 1$$

$$f = qz \quad \frac{\partial f}{\partial q} = z, \frac{\partial f}{\partial z} = q$$

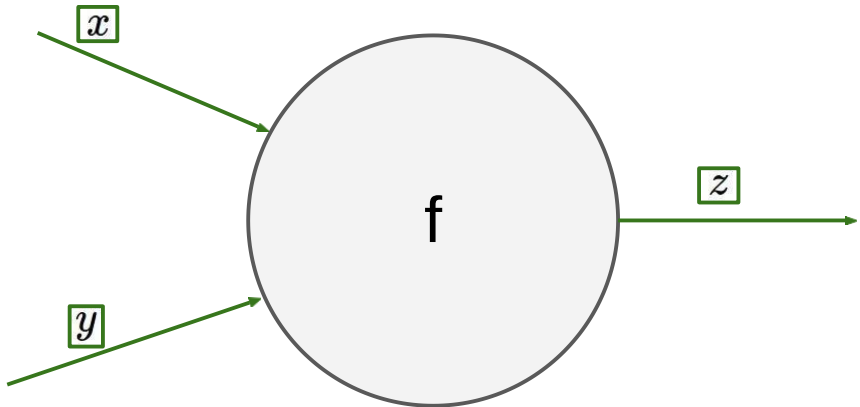
Want: $\frac{\partial f}{\partial x}, \frac{\partial f}{\partial y}, \frac{\partial f}{\partial z}$

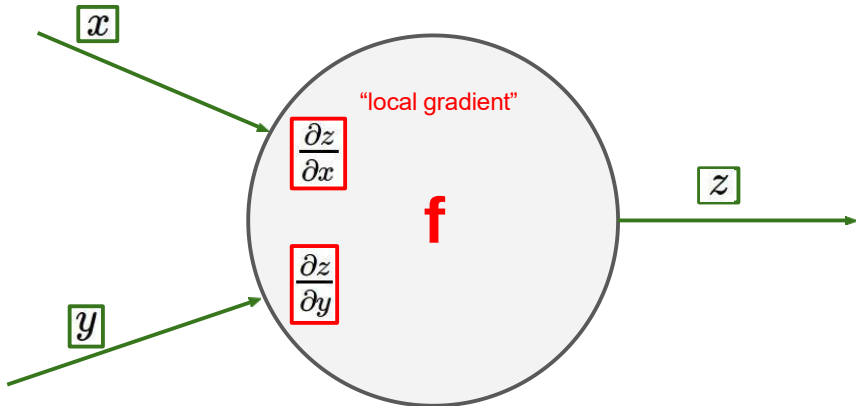


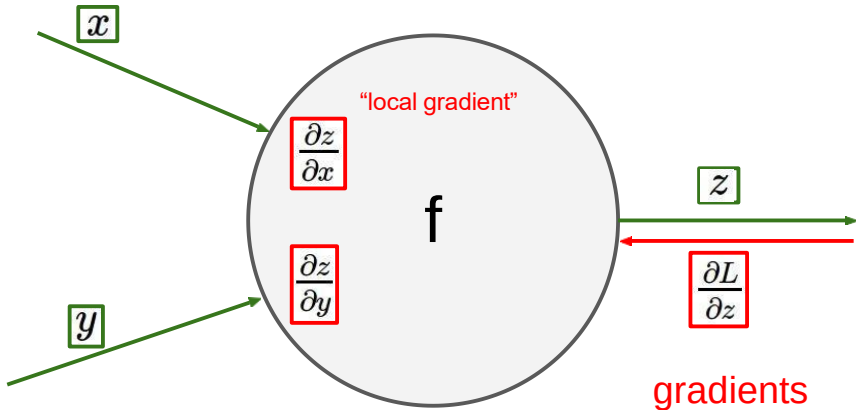
$$\frac{\partial f}{\partial x}$$

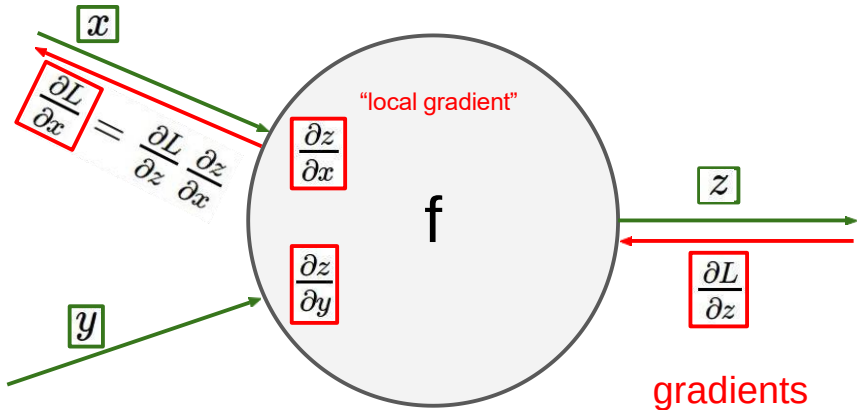
Chain rule:

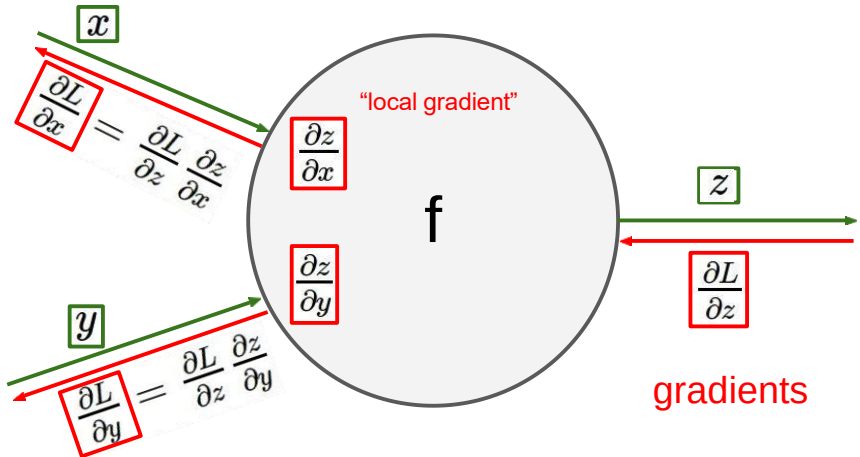
$$\frac{\partial f}{\partial x} = \frac{\partial f}{\partial q} \frac{\partial q}{\partial x}$$

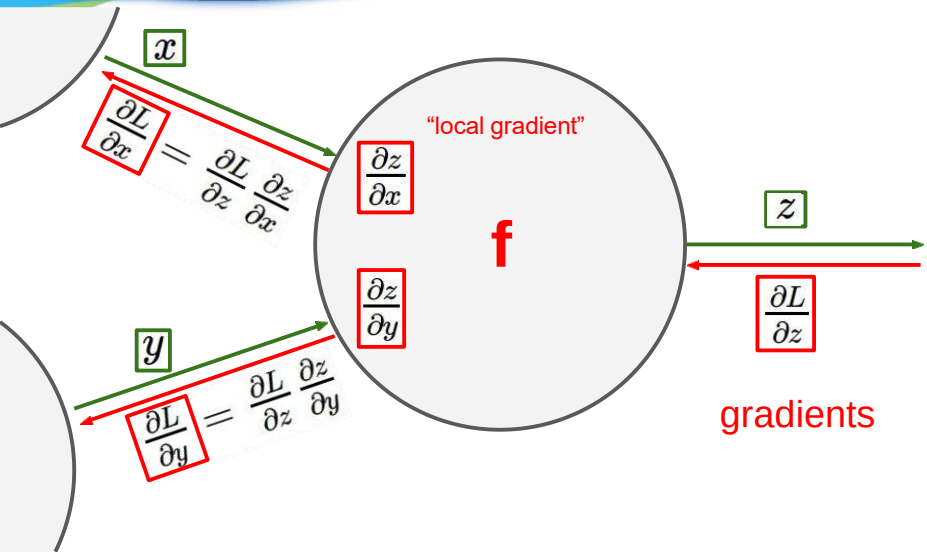






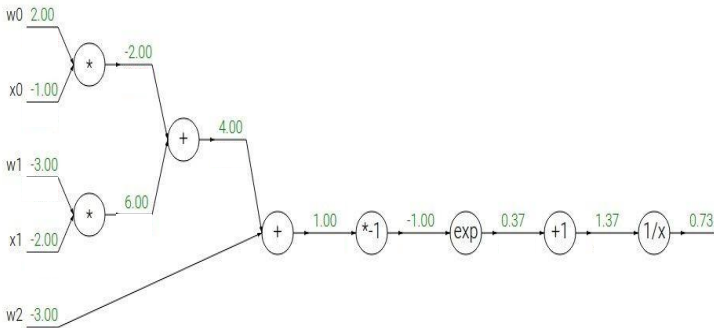






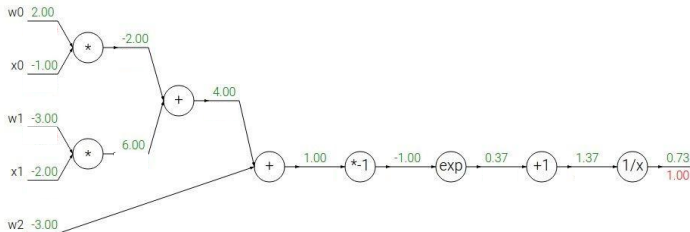
Another example:

$$f(w, x) = \frac{1}{1 + e^{-(w_0 x_0 + w_1 x_1 + w_2)}}$$



Another example:

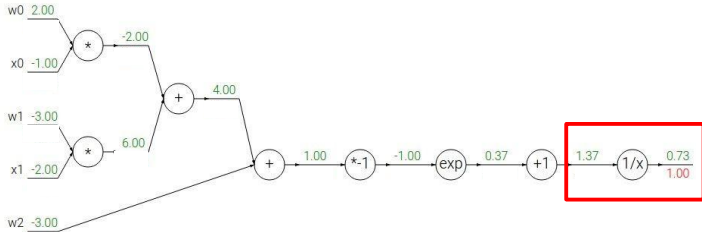
$$f(w, x) = \frac{1}{1 + e^{-(w_0 x_0 + w_1 x_1 + w_2)}}$$



$f(x) = e^x$	$\rightarrow$	$\frac{df}{dx} = e^x$		$f(x) = \frac{1}{x}$	$\rightarrow$	$\frac{df}{dx} = -1/x^2$
$f_a(x) = ax$	$\rightarrow$	$\frac{df}{dx} = a$		$f_c(x) = c + x$	$\rightarrow$	$\frac{df}{dx} = 1$

Another example:

$$f(w, x) = \frac{1}{1 + e^{-(w_0 x_0 + w_1 x_1 + w_2)}}$$



$$f(x) = e^x$$

→

$$\frac{df}{dx} = e^x$$

$$f_a(x) = ax$$

→

$$\frac{df}{dx} = a$$

$$f(x) = \frac{1}{x}$$

→

$$\frac{df}{dx} = -1/x^2$$

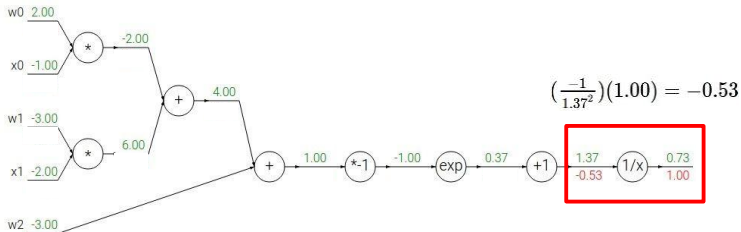
$$f_c(x) = c + x$$

→

$$\frac{df}{dx} = 1$$

Another example:

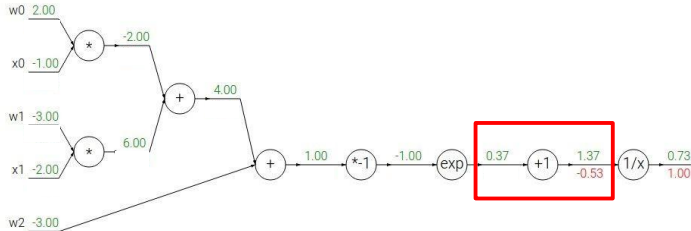
$$f(w, x) = \frac{1}{1 + e^{-(w_0x_0 + w_1x_1 + w_2x_2)}}$$



$f(x) = e^x \rightarrow \frac{df}{dx} = e^x$		$f(x) = \frac{1}{x} \rightarrow \frac{df}{dx} = -1/x^2$
$f_a(x) = ax \rightarrow \frac{df}{dx} = a$		$f_c(x) = c + x \rightarrow \frac{df}{dx} = 1$

Another example:

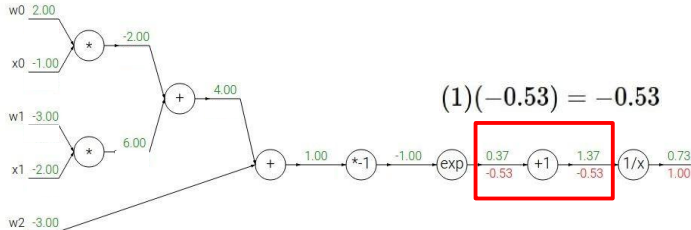
$$f(w, x) = \frac{1}{1 + e^{-(w_0 x_0 + w_1 x_1 + w_2)}}$$



$f(x) = e^x$	$\rightarrow$	$\frac{df}{dx} = e^x$		$f(x) = \frac{1}{x}$	$\rightarrow$	$\frac{df}{dx} = -1/x^2$
$f_a(x) = ax$	$\rightarrow$	$\frac{df}{dx} = a$		$f_c(x) = c + x$	$\rightarrow$	$\frac{df}{dx} = 1$

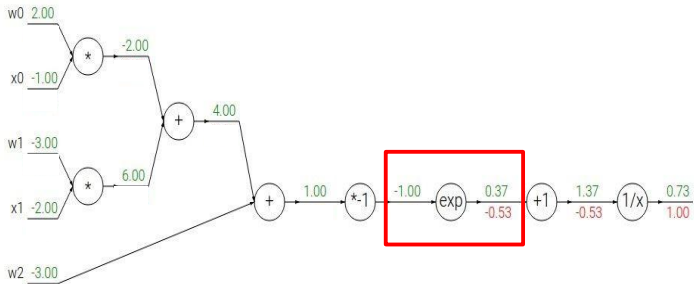
Another example:

$$f(w, x) = \frac{1}{1 + e^{-(w_0 x_0 + w_1 x_1 + w_2 x_2)}}$$



$f(x) = e^x$	$\rightarrow$	$\frac{df}{dx} = e^x$		$f(x) = \frac{1}{x}$	$\rightarrow$	$\frac{df}{dx} = -1/x^2$
$f_a(x) = ax$	$\rightarrow$	$\frac{df}{dx} = a$		$f_c(x) = c + x$	$\rightarrow$	$\frac{df}{dx} = 1$

Another example:



$$f(x) = e^x \rightarrow \frac{df}{dx} = e^x$$

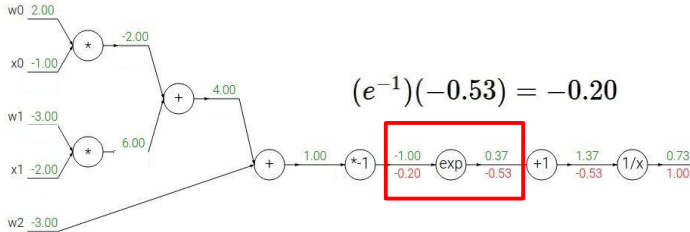
$$f_a(x) = ax \rightarrow \frac{df}{dx} = a$$

$$f(x) = \frac{1}{x} \rightarrow \frac{df}{dx} = -1/x^2$$

$$f_c(x) = c + x \rightarrow \frac{df}{dx} = 1$$

Another example:

$$f(w, x) = \frac{1}{1 + e^{-(w_0 x_0 + w_1 x_1 + w_2 x_2)}}$$



$$f(x) = e^x \rightarrow \frac{df}{dx} = e^x$$

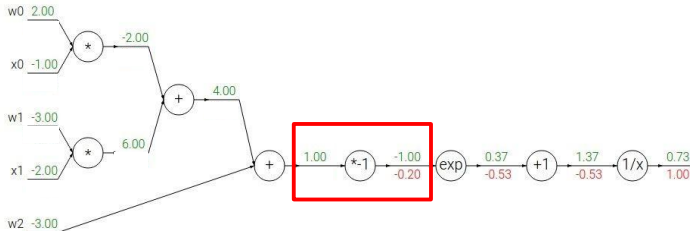
$$f_a(x) = ax \rightarrow \frac{df}{dx} = a$$

$$f(x) = \frac{1}{x} \rightarrow \frac{df}{dx} = -1/x^2$$

$$f_c(x) = c + x \rightarrow \frac{df}{dx} = 1$$

Another example:

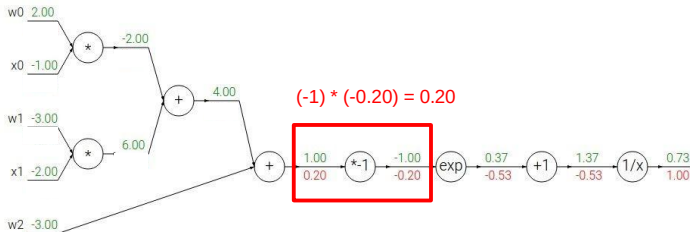
$$f(w, x) = \frac{1}{1 + e^{-(w_0x_0 + w_1x_1 + w_2x_2)}}$$



$f(x) = e^x$	$\rightarrow$	$\frac{df}{dx} = e^x$		$f(x) = \frac{1}{x}$	$\rightarrow$	$\frac{df}{dx} = -1/x^2$
$f_a(x) = ax$	$\rightarrow$	$\frac{df}{dx} = a$		$f_c(x) = c + x$	$\rightarrow$	$\frac{df}{dx} = 1$

Another example:

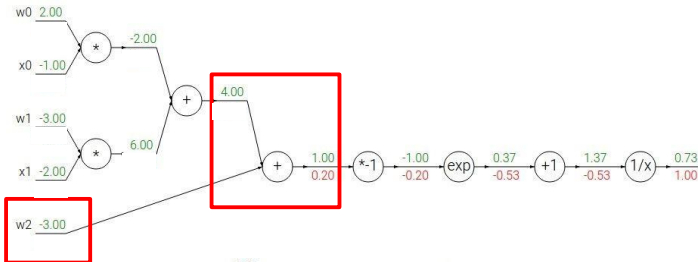
$$f(w, x) = \frac{1}{1 + e^{-(w_0x_0 + w_1x_1 + w_2x_2)}}$$



$f(x) = e^x \rightarrow \frac{df}{dx} = e^x$		$f(x) = \frac{1}{x} \rightarrow \frac{df}{dx} = -1/x^2$
$f_a(x) = ax \rightarrow \frac{df}{dx} = a$		$f_c(x) = c + x \rightarrow \frac{df}{dx} = 1$

Another example:

$$f(w, x) = \frac{1}{1 + e^{-(w_0x_0 + w_1x_1 + w_2x_2)}}$$



$$f(x) = e^x$$

→

$$\frac{df}{dx} = e^x$$

$$f(x) = \frac{1}{x}$$

→

$$\frac{df}{dx} = -1/x^2$$

$$f_a(x) = ax$$

→

$$\frac{df}{dx} = a$$

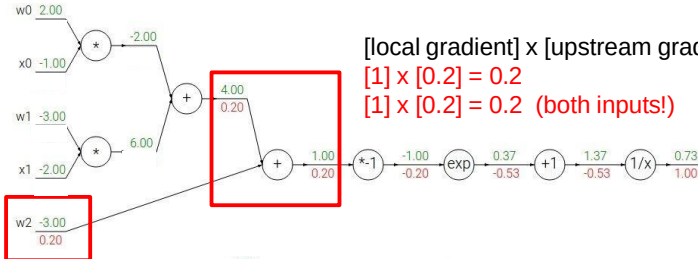
$$f_c(x) = c + x$$

→

$$\frac{df}{dx} = 1$$

Another example:

$$f(w, x) = \frac{1}{1 + e^{-(w_0 x_0 + w_1 x_1 + w_2)}}$$



$$f(x) = e^x$$

→

$$\frac{df}{dx} = e^x$$

$$f(x) = \frac{1}{x}$$

→

$$\frac{df}{dx} = -1/x^2$$

$$f_a(x) = ax$$

→

$$\frac{df}{dx} = a$$

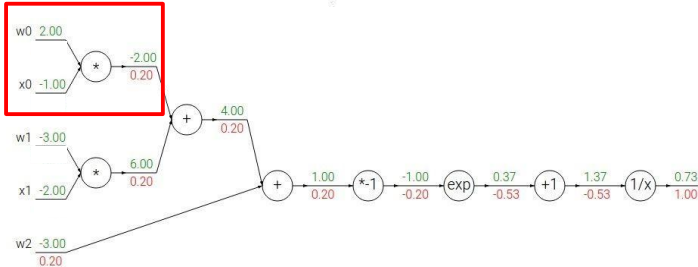
$$f_c(x) = c + x$$

→

$$\frac{df}{dx} = 1$$

Another example:

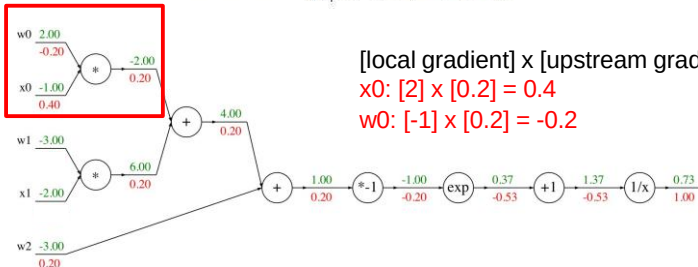
$$f(w, x) = \frac{1}{1 + e^{-(w_0x_0 + w_1x_1 + w_2x_2)}}$$



$f(x) = e^x$	$\rightarrow$	$\frac{df}{dx} = e^x$		$f(x) = \frac{1}{x}$	$\rightarrow$	$\frac{df}{dx} = -1/x^2$
$f_a(x) = ax$	$\rightarrow$	$\frac{df}{dx} = a$		$f_c(x) = c + x$	$\rightarrow$	$\frac{df}{dx} = 1$

Another example:

$$f(w, x) = \frac{1}{1 + e^{-(w_0 x_0 + w_1 x_1 + w_2 x_2)}}$$



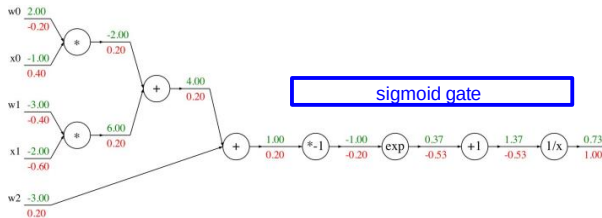
$f(x) = e^x$	$\rightarrow$	$\frac{df}{dx} = e^x$		$f(x) = \frac{1}{x}$	$\rightarrow$	$\frac{df}{dx} = -1/x^2$
$f_a(x) = ax$	$\rightarrow$	$\frac{df}{dx} = a$		$f_c(x) = c + x$	$\rightarrow$	$\frac{df}{dx} = 1$

$$f(w, x) = \frac{1}{1 + e^{-(w_0 x_0 + w_1 x_1 + w_2 x_2)}}$$

$$\sigma(x) = \frac{1}{1 + e^{-x}}$$

sigmoid function

$$\frac{d\sigma(x)}{dx} = \frac{e^{-x}}{(1 + e^{-x})^2} = \left(\frac{1 + e^{-x} - 1}{1 + e^{-x}} \right) \left(\frac{1}{1 + e^{-x}} \right) = (1 - \sigma(x)) \sigma(x)$$

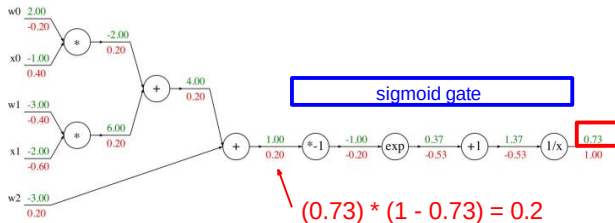


$$f(w, x) = \frac{1}{1 + e^{-(w_0 x_0 + w_1 x_1 + w_2)}}$$

$$\sigma(x) = \frac{1}{1 + e^{-x}}$$

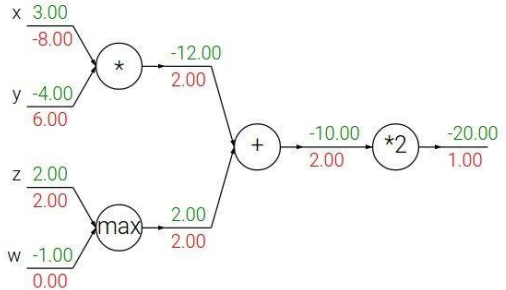
sigmoid function

$$\frac{d\sigma(x)}{dx} = \frac{e^{-x}}{(1 + e^{-x})^2} = \left(\frac{1 + e^{-x} - 1}{1 + e^{-x}} \right) \left(\frac{1}{1 + e^{-x}} \right) = (1 - \sigma(x)) \sigma(x)$$



Patterns in backward flow

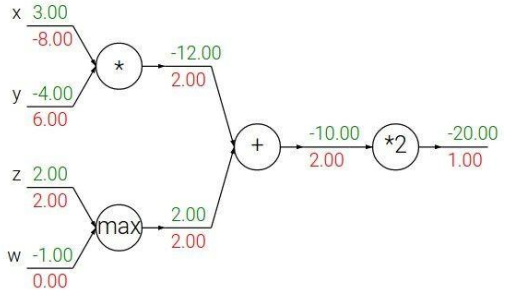
add gate: gradient distributor



Patterns in backward flow

add gate: gradient distributor

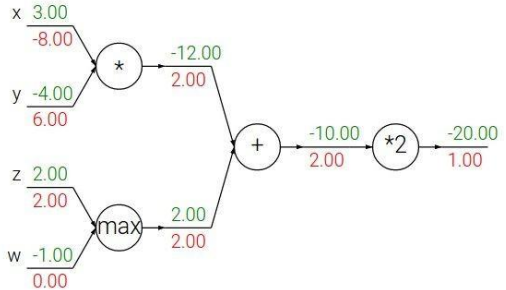
Q: What is a **max** gate?



Patterns in backward flow

add gate: gradient distributor

max gate: gradient router

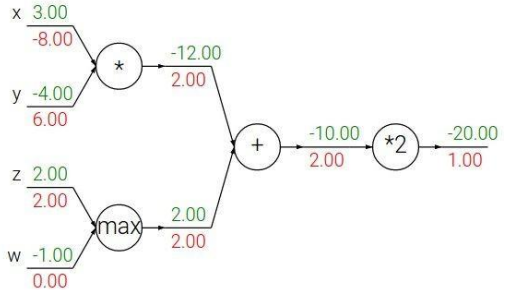


Patterns in backward flow

add gate: gradient distributor

max gate: gradient router

Q: What is a **mul** gate?

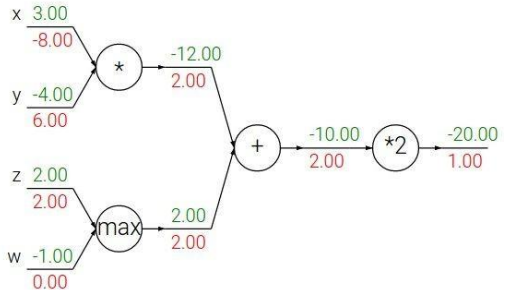


Patterns in backward flow

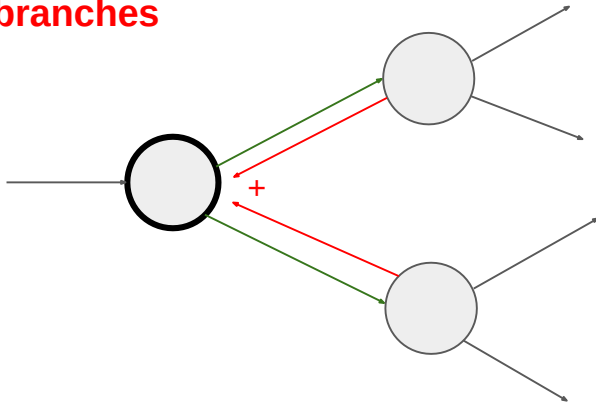
add gate: gradient distributor

max gate: gradient router

mul gate: gradient switcher



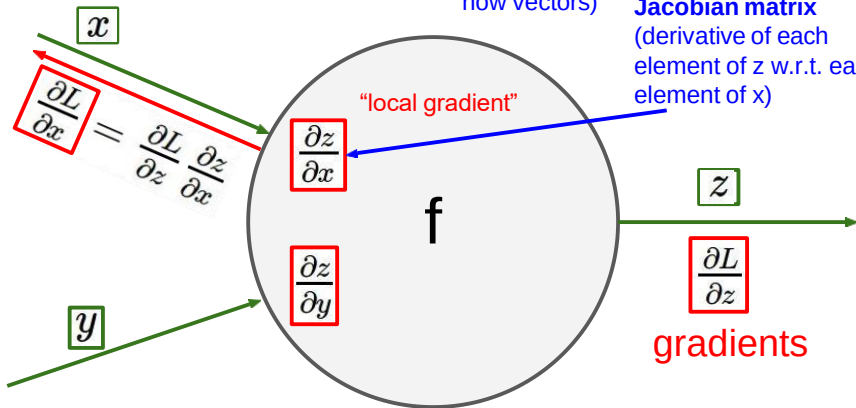
Gradients add at branches



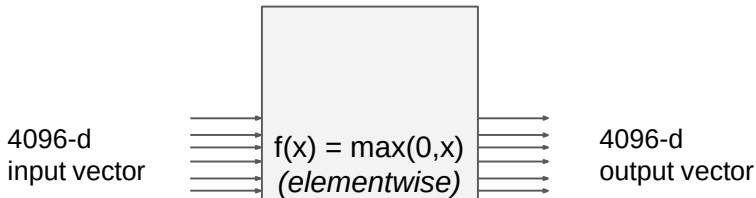
Gradients for vectorized code

(x,y,z are
now vectors)

This is now the
Jacobian matrix
(derivative of each
element of z w.r.t. each
element of x)



Vectorized operations

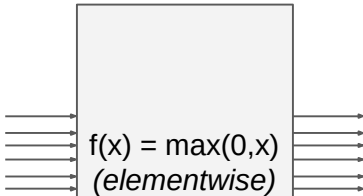


Vectorized operations

$$\frac{\partial L}{\partial x} = \boxed{\frac{\partial f}{\partial x}} \frac{\partial L}{\partial f}$$

Jacobian matrix

4096-d
input vector



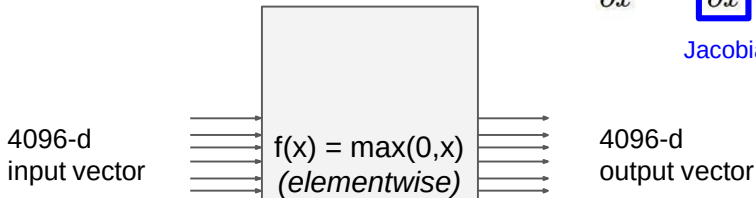
4096-d
output vector

Q: what is the
size of the
Jacobian matrix?

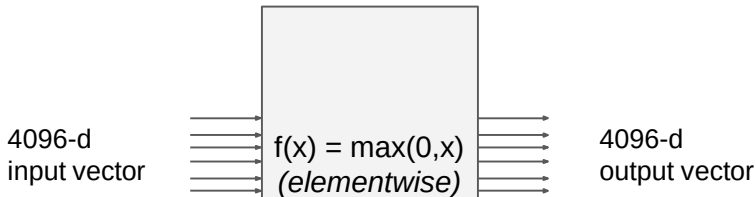
Vectorized operations

$$\frac{\partial L}{\partial x} = \boxed{\frac{\partial f}{\partial x}} \frac{\partial L}{\partial f}$$

Jacobian matrix



Q: what is the
size of the
Jacobian matrix?
[4096 x 4096!]



Q: what is the size of the Jacobian matrix?
[4096 x 4096!]

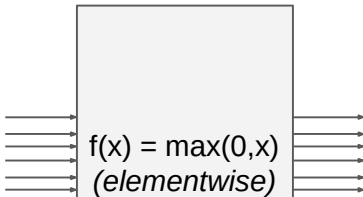
in practice we process an entire minibatch (e.g. 100) of examples at one time:
i.e. Jacobian would technically be a [409,600 x 409,600] matrix :)

Vectorized operations

$$\frac{\partial L}{\partial x} = \boxed{\frac{\partial f}{\partial x}} \frac{\partial L}{\partial f}$$

Jacobian matrix

4096-d
input vector



4096-d
output vector

Q: what is the
size of the
Jacobian matrix?
[4096 x 4096!]

Q2: what does it
look like?

A vectorized example:

$$f(x, W) = \|W \cdot x\|^2 = \sum_{i=1}^n (W \cdot x)_i^2$$

A vectorized example:

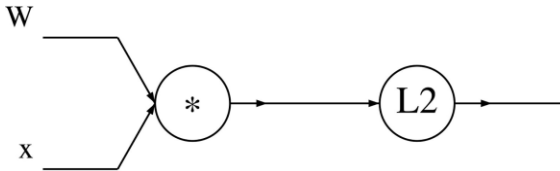
$$f(x, W) = \|W \cdot x\|^2 = \sum_{i=1}^n (W \cdot x)_i^2$$

$\Downarrow \quad \Downarrow$

$$\in \mathbb{R}^n \quad \in \mathbb{R}^{n \times n}$$

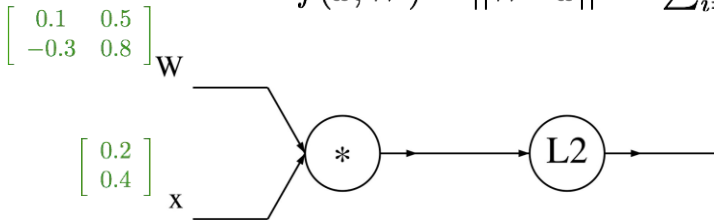
A vectorized example:

$$f(x, W) = ||W \cdot x||^2 = \sum_{i=1}^n (W \cdot x)_i^2$$



A vectorized example:

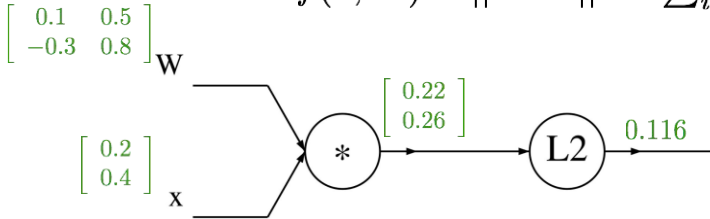
$$f(x, W) = ||W \cdot x||^2 = \sum_{i=1}^n (W \cdot x)_i^2$$



$$q = W \cdot x = \begin{pmatrix} W_{1,1}x_1 + \cdots + W_{1,n}x_n \\ \vdots \\ W_{n,1}x_1 + \cdots + W_{n,n}x_n \end{pmatrix}$$
$$f(q) = ||q||^2 = q_1^2 + \cdots + q_n^2$$

A vectorized example:

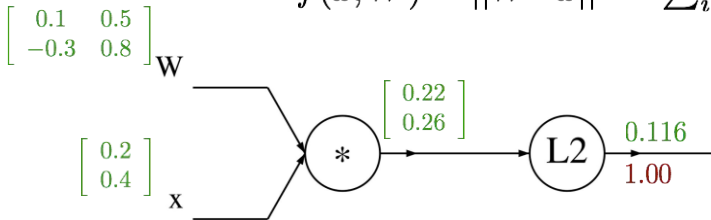
$$f(x, W) = ||W \cdot x||^2 = \sum_{i=1}^n (W \cdot x)_i^2$$



$$q = W \cdot x = \begin{pmatrix} W_{1,1}x_1 + \cdots + W_{1,n}x_n \\ \vdots \\ W_{n,1}x_1 + \cdots + W_{n,n}x_n \end{pmatrix}$$
$$f(q) = ||q||^2 = q_1^2 + \cdots + q_n^2$$

A vectorized example:

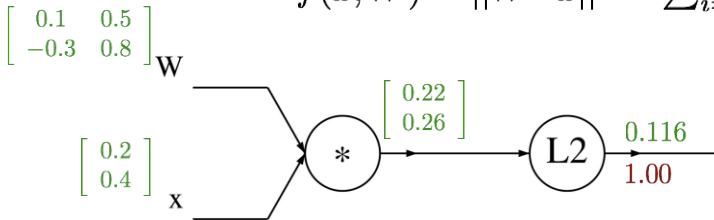
$$f(x, W) = ||W \cdot x||^2 = \sum_{i=1}^n (W \cdot x)_i^2$$



$$q = W \cdot x = \begin{pmatrix} W_{1,1}x_1 + \cdots + W_{1,n}x_n \\ \vdots \\ W_{n,1}x_1 + \cdots + W_{n,n}x_n \end{pmatrix}$$
$$f(q) = ||q||^2 = q_1^2 + \cdots + q_n^2$$

A vectorized example:

$$f(x, W) = ||W \cdot x||^2 = \sum_{i=1}^n (W \cdot x)_i^2$$



$$q = W \cdot x = \begin{pmatrix} W_{1,1}x_1 + \cdots + W_{1,n}x_n \\ \vdots \\ W_{n,1}x_1 + \cdots + W_{n,n}x_n \end{pmatrix}$$

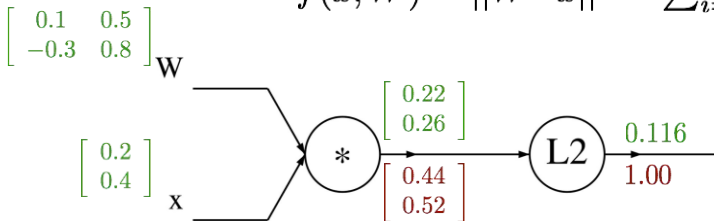
$$f(q) = ||q||^2 = q_1^2 + \cdots + q_n^2$$

$$\frac{\partial f}{\partial q_i} = 2q_i$$

$$\nabla_q f = 2q$$

A vectorized example:

$$f(x, W) = ||W \cdot x||^2 = \sum_{i=1}^n (W \cdot x)_i^2$$



$$q = W \cdot x = \begin{pmatrix} W_{1,1}x_1 + \cdots + W_{1,n}x_n \\ \vdots \\ W_{n,1}x_1 + \cdots + W_{n,n}x_n \end{pmatrix}$$

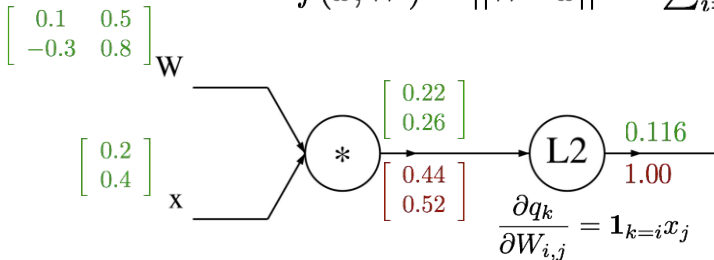
$$f(q) = ||q||^2 = q_1^2 + \cdots + q_n^2$$

$$\frac{\partial f}{\partial q_i} = 2q_i$$

$$\nabla_q f = 2q$$

A vectorized example:

$$f(x, W) = ||W \cdot x||^2 = \sum_{i=1}^n (W \cdot x)_i^2$$

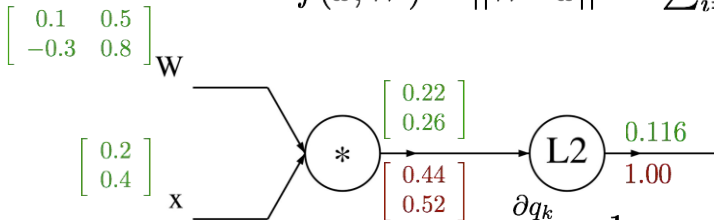


$$q = W \cdot x = \begin{pmatrix} W_{1,1}x_1 + \cdots + W_{1,n}x_n \\ \vdots \\ W_{n,1}x_1 + \cdots + W_{n,n}x_n \end{pmatrix}$$

$$f(q) = ||q||^2 = q_1^2 + \cdots + q_n^2$$

A vectorized example:

$$f(x, W) = ||W \cdot x||^2 = \sum_{i=1}^n (W \cdot x)_i^2$$



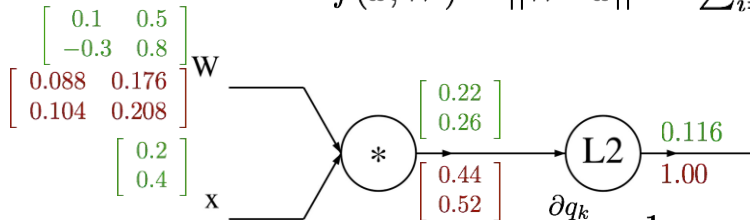
$$\begin{aligned}
 \frac{\partial q_k}{\partial W_{i,j}} &= \mathbf{1}_{k=i} x_j \\
 \frac{\partial f}{\partial W_{i,j}} &= \sum_k \frac{\partial f}{\partial q_k} \frac{\partial q_k}{\partial W_{i,j}} \\
 &= \sum_k (2q_k) (\mathbf{1}_{k=i} x_j) \\
 &= 2q_i x_j
 \end{aligned}$$

$$q = W \cdot x = \begin{pmatrix} W_{1,1}x_1 + \cdots + W_{1,n}x_n \\ \vdots \\ W_{n,1}x_1 + \cdots + W_{n,n}x_n \end{pmatrix}$$

$$f(q) = ||q||^2 = q_1^2 + \cdots + q_n^2$$

A vectorized example:

$$f(x, W) = ||W \cdot x||^2 = \sum_{i=1}^n (W \cdot x)_i^2$$



$$q = W \cdot x = \begin{pmatrix} W_{1,1}x_1 + \cdots + W_{1,n}x_n \\ \vdots \\ W_{n,1}x_1 + \cdots + W_{n,n}x_n \end{pmatrix}$$

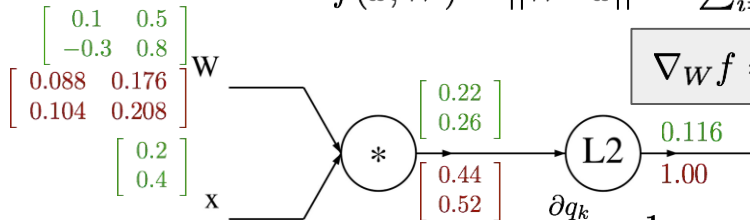
$$f(q) = ||q||^2 = q_1^2 + \cdots + q_n^2$$

$$\frac{\partial q_k}{\partial W_{i,j}} = \mathbf{1}_{k=i}x_j$$

$$\begin{aligned}
 \frac{\partial f}{\partial W_{i,j}} &= \sum_k \frac{\partial f}{\partial q_k} \frac{\partial q_k}{\partial W_{i,j}} \\
 &= \sum_k (2q_k)(\mathbf{1}_{k=i}x_j) \\
 &= 2q_i x_j
 \end{aligned}$$

A vectorized example:

$$f(x, W) = ||W \cdot x||^2 = \sum_{i=1}^n (W \cdot x)_i^2$$



$$\nabla_W f = 2q \cdot x^T$$

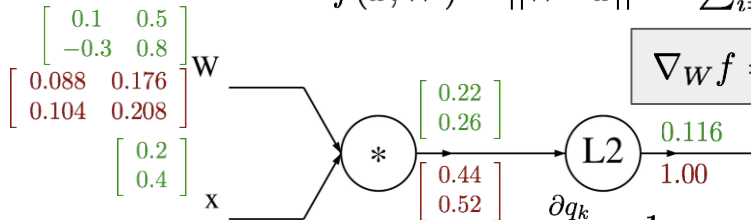
$$\begin{aligned}
 \frac{\partial q_k}{\partial W_{i,j}} &= \mathbf{1}_{k=i} x_j \\
 \frac{\partial f}{\partial W_{i,j}} &= \sum_k \frac{\partial f}{\partial q_k} \frac{\partial q_k}{\partial W_{i,j}} \\
 &= \sum_k (2q_k) (\mathbf{1}_{k=i} x_j) \\
 &= 2q_i x_j
 \end{aligned}$$

$$q = W \cdot x = \begin{pmatrix} W_{1,1}x_1 + \dots + W_{1,n}x_n \\ \vdots \\ W_{n,1}x_1 + \dots + W_{n,n}x_n \end{pmatrix}$$

$$f(q) = ||q||^2 = q_1^2 + \dots + q_n^2$$

A vectorized example:

$$f(x, W) = ||W \cdot x||^2 = \sum_{i=1}^n (W \cdot x)_i^2$$



$$\nabla_W f = 2q \cdot x^T$$

Always check: The gradient with respect to a variable should have the same shape as the variable

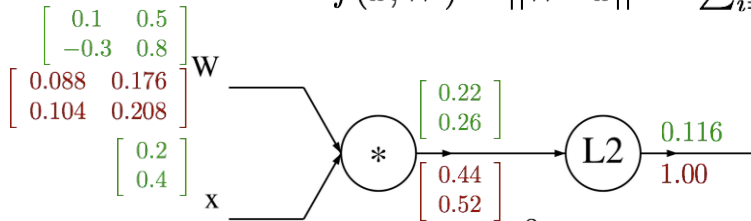
$$\begin{aligned}
 \frac{\partial q_k}{\partial W_{i,j}} &= \mathbf{1}_{k=i} x_j \\
 \frac{\partial f}{\partial W_{i,j}} &= \sum_k \frac{\partial f}{\partial q_k} \frac{\partial q_k}{\partial W_{i,j}} \\
 &= \sum_k (2q_k) (\mathbf{1}_{k=i} x_j) \\
 &= 2q_i x_j
 \end{aligned}$$

$$q = W \cdot x = \begin{pmatrix} W_{1,1}x_1 + \dots + W_{1,n}x_n \\ \vdots \\ W_{n,1}x_1 + \dots + W_{n,n}x_n \end{pmatrix}$$

$$f(q) = ||q||^2 = q_1^2 + \dots + q_n^2$$

A vectorized example:

$$f(x, W) = ||W \cdot x||^2 = \sum_{i=1}^n (W \cdot x)_i^2$$



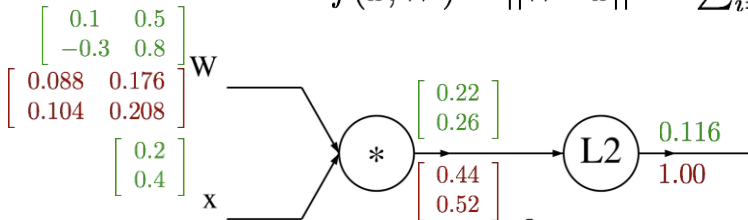
$$\frac{\partial q_k}{\partial x_i} = W_{k,i}$$

$$q = W \cdot x = \begin{pmatrix} W_{1,1}x_1 + \cdots + W_{1,n}x_n \\ \vdots \\ W_{n,1}x_1 + \cdots + W_{n,n}x_n \end{pmatrix}$$

$$f(q) = ||q||^2 = q_1^2 + \cdots + q_n^2$$

A vectorized example:

$$f(x, W) = ||W \cdot x||^2 = \sum_{i=1}^n (W \cdot x)_i^2$$



$$q = W \cdot x = \begin{pmatrix} W_{1,1}x_1 + \cdots + W_{1,n}x_n \\ \vdots \\ W_{n,1}x_1 + \cdots + W_{n,n}x_n \end{pmatrix}$$

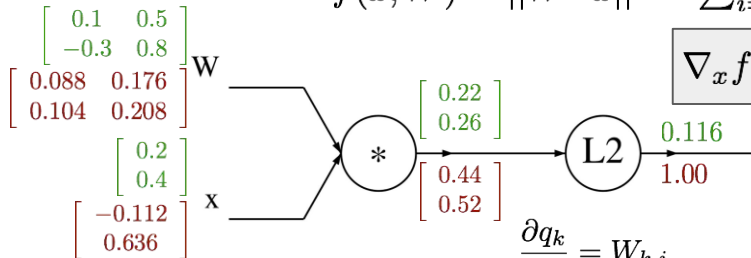
$$f(q) = ||q||^2 = q_1^2 + \cdots + q_n^2$$

$$\begin{aligned} \frac{\partial q_k}{\partial x_i} &= W_{k,i} \\ \frac{\partial f}{\partial x_i} &= \sum_k \frac{\partial f}{\partial q_k} \frac{\partial q_k}{\partial x_i} \\ &= \sum_k 2q_k W_{k,i} \end{aligned}$$

A vectorized example:

$$f(x, W) = ||W \cdot x||^2 = \sum_{i=1}^n (W \cdot x)_i^2$$

$$\nabla_x f = 2W^T \cdot q$$



$$q = W \cdot x = \begin{pmatrix} W_{1,1}x_1 + \dots + W_{1,n}x_n \\ \vdots \\ W_{n,1}x_1 + \dots + W_{n,n}x_n \end{pmatrix}$$

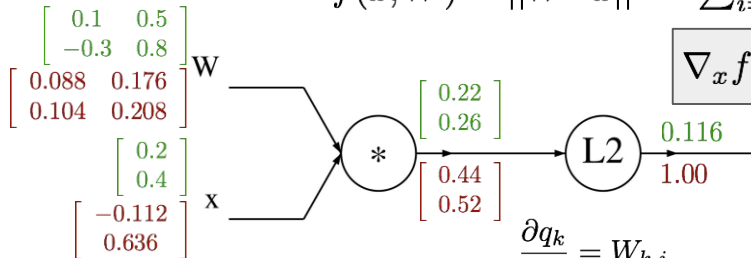
$$f(q) = ||q||^2 = q_1^2 + \dots + q_n^2$$

$$\begin{aligned} \frac{\partial q_k}{\partial x_i} &= W_{k,i} \\ \frac{\partial f}{\partial x_i} &= \sum_k \frac{\partial f}{\partial q_k} \frac{\partial q_k}{\partial x_i} \\ &= \sum_k 2q_k W_{k,i} \end{aligned}$$

A vectorized example:

$$f(x, W) = ||W \cdot x||^2 = \sum_{i=1}^n (W \cdot x)_i^2$$

$$\nabla_x f = 2W^T \cdot q$$

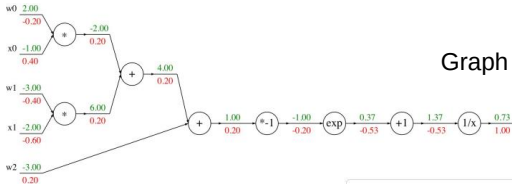


$$q = W \cdot x = \begin{pmatrix} W_{1,1}x_1 + \dots + W_{1,n}x_n \\ \vdots \\ W_{n,1}x_1 + \dots + W_{n,n}x_n \end{pmatrix}$$

$$f(q) = ||q||^2 = q_1^2 + \dots + q_n^2$$

$$\begin{aligned} \frac{\partial q_k}{\partial x_i} &= W_{k,i} \\ \frac{\partial f}{\partial x_i} &= \sum_k \frac{\partial f}{\partial q_k} \frac{\partial q_k}{\partial x_i} \\ &= \sum_k 2q_k W_{k,i} \end{aligned}$$

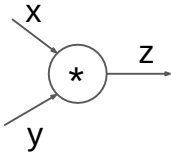
Modularized implementation: forward / backward API



Graph (or Net) object (*rough psuedo code*)

```
class ComputationalGraph(object):
    #...
    def forward(inputs):
        # 1. [pass inputs to input gates...]
        # 2. forward the computational graph:
        for gate in self.graph.nodes_topologically_sorted():
            gate.forward()
        return loss # the final gate in the graph outputs the loss
    def backward():
        for gate in reversed(self.graph.nodes_topologically_sorted()):
            gate.backward() # little piece of backprop (chain rule applied)
        return inputs_gradients
```

Modularized implementation: forward / backward API



(x,y,z are scalars)

```
class MultiplyGate(object):
```

```
    def forward(x,y):
```

```
        z = x*y
```

```
        return z
```

```
    def backward(dz):
```

```
        # dx = ... #todo
```

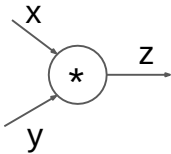
```
        # dy = ... #todo
```

```
        return [dx, dy]
```

$$\frac{\partial L}{\partial z}$$

$$\frac{\partial L}{\partial x}$$

Modularized implementation: forward / backward API



(x,y,z are scalars)

```
class MultiplyGate(object):
    def forward(x,y):
        z = x*y
        self.x = x # must keep these around!
        self.y = y
        return z
    def backward(dz):
        dx = self.y * dz # [dz/dx * dL/dz]
        dy = self.x * dz # [dz/dy * dL/dz]
        return [dx, dy]
```

Example: Caffe

Search master	caffe / src / caffe / layers /	Create new file	Upload file	Find file	History
shelhamer committed on GitHub Merge pull request #4630 from BGeneload/duff15_1k · 1k					
Latest commit v467a75 21 days ago					
absext_layer.cpp	diamond layer headers	a year ago			
absext_layer.cu	diamond layer headers	a year ago			
accuracy_layer.cpp	diamond layer headers	a year ago			
argmax_layer.cpp	diamond layer headers	a year ago			
base_conv_layer.cpp	enable dilated convolution	a year ago			
base_data_layer.cpp	Using default from proto for prefetch	3 months ago			
base_data_layer.cu	Switched multi-GPU to NCCL	3 months ago			
batch_norm_layer.cpp	Add missing spaces besides equal signs in batch_norm_layer.cpp	4 months ago			
batch_norm_layer.cu	diamond layer headers	a year ago			
batch_relu_layer.cpp	diamond layer headers	a year ago			
batch_relu_layer.cu	diamond layer headers	a year ago			
bias_layer.cpp	Remove incorrect cast of gemm in arg to Dtype in BiasLayer	a year ago			
bias_layer.cu	Separation and generalization of ChannelwiseAffineLayer into BiasLayer	a year ago			
bifrost_layer.cpp	diamond layer headers	a year ago			
bifrost_layer.cu	diamond layer headers	a year ago			
concat_layer.cpp	diamond layer headers	a year ago			
concat_layer.cu	diamond layer headers	a year ago			
contrastive_loss_layer.cpp	diamond layer headers	a year ago			
contrastive_loss_layer.cu	diamond layer headers	a year ago			
conv_layer.cpp	Add support for 2D dilated convolution	a year ago			
conv_layer.cu	diamond layer headers	a year ago			
crop_layer.cpp	remove redundant operations in Crop layer (#5138)	2 months ago			
crop_layer.cu	remove redundant operations in Crop layer (#5138)	2 months ago			
cuDNN_conv_layer.cpp	diamond layer headers	a year ago			
cuDNN_conv_layer.cu	Add cuDNN v5 support, drop cuDNN v3 support	11 months ago			

cuDNN_fc_layer.cpp	diamond layer headers	a year ago
cuDNN_fc_layer.cu	diamond layer headers	a year ago
cuDNN_fm_layer.cpp	diamond layer headers	a year ago
cuDNN_fm_layer.cu	diamond layer headers	a year ago
cuDNN_pooling_layer.cpp	diamond layer headers	a year ago
cuDNN_pooling_layer.cu	diamond layer headers	a year ago
cuDNN_relu_layer.cpp	Add cuDNN v5 support, drop cuDNN v3 support	11 months ago
cuDNN_relu_layer.cu	Add cuDNN v5 support, drop cuDNN v3 support	11 months ago
cuDNN_sigmoid_layer.cpp	Add cuDNN v5 support, drop cuDNN v3 support	11 months ago
cuDNN_sigmoid_layer.cu	Add cuDNN v5 support, drop cuDNN v3 support	11 months ago
cuDNN_softmax_layer.cpp	diamond layer headers	a year ago
cuDNN_softmax_layer.cu	diamond layer headers	a year ago
cuDNN_spatial_avg.cpp	Add cuDNN v5 support, drop cuDNN v3 support	11 months ago
cuDNN_spatial_avg.cu	Add cuDNN v5 support, drop cuDNN v3 support	11 months ago
data_layer.cpp	Switched multi-GPU to NCCL	3 months ago
deconv_layer.cpp	enable dilated deconvolution	a year ago
deconv_layer.cu	diamond layer headers	a year ago
dropout_layer.cpp	supporting 1D blocks in Dropout layer headers	a year ago
dropout_layer.cu	diamond layer headers	a year ago
dummy_data_layer.cpp	diamond layer headers	a year ago
eltwise_layer.cpp	diamond layer headers	a year ago
eltwise_layer.cu	diamond layer headers	a year ago
elu_layer.cpp	ELU layer with basic tests	a year ago
elu_layer.cu	ELU layer with basic tests	a year ago
embed_layer.cpp	diamond layer headers	a year ago
embed_layer.cu	diamond layer headers	a year ago
euclidean_loss_layer.cpp	diamond layer headers	a year ago
euclidean_loss_layer.cu	diamond layer headers	a year ago
exp_layer.cpp	Solving issue with exp layer with base e	a year ago
exp_layer.cu	diamond layer headers	a year ago

Caffe is licensed under [BSD 2-Clause](#)

Caffe Sigmoid Layer

```

1  #include <cmath>
2  #include <vector>
3
4  #include "caffe/layers/sgmoid_layer.hpp"
5
6  namespace caffe {
7
8  template <typename Dtype>
9  inline Dtype sigmoid(Dtype x) {
10     return 1. / (1. + exp(-x));
11 }
12
13 template <typename Dtype>
14 void SigmoidLayer<Dtype>::Forward_cpu(const vector<Blob<Dtype>*>& bottom,
15     const vector<Blob<Dtype>*>& top) {
16     const Dtype* bottom_data = bottom[0]->cpu_data();
17     Dtype* top_data = top[0]->mutable_cpu_data();
18     const int count = bottom[0]->count();
19     for (int i = 0; i < count; ++i) {
20         top_data[i] = sigmoid(bottom_data[i]);
21     }
22 }
23
24 template <typename Dtype>
25 void SigmoidLayer<Dtype>::Backward_cpu(const vector<Blob<Dtype>*>& top,
26     const vector<bool>& propagate_down,
27     const vector<Blob<Dtype>*>& bottom) {
28     if (propagate_down[0]) {
29         const Dtype* top_data = top[0]->cpu_data();
30         const Dtype* top_diff = top[0]->cpu_diff();
31         Dtype* bottom_diff = bottom[0]->mutable_cpu_diff();
32         const int count = bottom[0]->count();
33         for (int i = 0; i < count; ++i) {
34             const Dtype sigmoid_x = top_data[i];
35             bottom_diff[i] = top_diff[i] * sigmoid_x * (1. - sigmoid_x);
36         }
37     }
38 }
39
40 #ifdef CPU_ONLY
41 STUB_GPU(SigmoidLayer);
42 #endif
43
44 INSTANTIATE_CLASS(SigmoidLayer);
45
46 } // namespace caffe

```

$$\sigma(x) = \frac{1}{1 + e^{-x}}$$

* top_diff (chain rule)

$$(1 - \sigma(x)) \sigma(x)$$

Summary so far...

- neural nets will be very large: impractical to write down gradient formula by hand for all parameters
- **backpropagation** = recursive application of the chain rule along a computational graph to compute the gradients of all inputs/parameters/intermediates
- implementations maintain a graph structure, where the nodes implement the **forward()** / **backward()** API
- **forward**: compute result of an operation and save any intermediates needed for gradient computation in memory
- **backward**: apply the chain rule to compute the gradient of the loss function with respect to the inputs

Next: Neural Networks

Neural networks: without the brain stuff

(Before) Linear score function: $f = Wx$

Neural networks: without the brain stuff

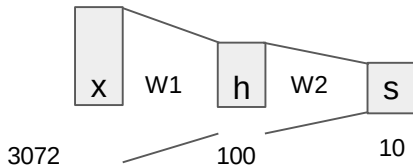
(**Before**) Linear score function: $f = Wx$

(**Now**) 2-layer Neural Network $f = W_2 \max(0, W_1 x)$

Neural networks: without the brain stuff

(Before) Linear score function: (Now) 2-layer

Neural Network



$$f = Wx$$

$$f = W_2 \max(0, W_1 x)$$

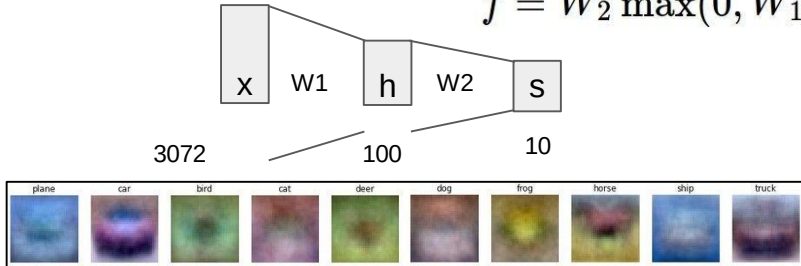
Neural networks: without the brain stuff

Before) Linear score function: (Now) 2-layer

Neural Network

$$f = Wx$$

$$f = W_2 \max(0, W_1 x)$$



Neural networks: without the brain stuff

(**Before**) Linear score function: $f = Wx$

(**Now**) 2-layer Neural Network $f = W_2 \max(0, W_1 x)$
or 3-layer Neural Network

$$f = W_3 \max(0, W_2 \max(0, W_1 x))$$

Full implementation of training a 2-layer Neural Network needs ~20 lines:

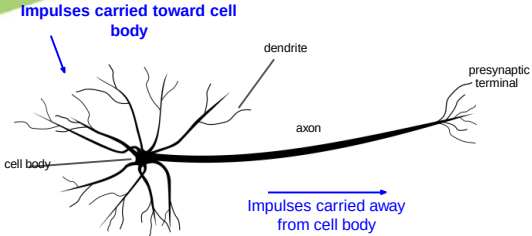
```
1 import numpy as np
2 from numpy.random import randn
3
4 N, D_in, H, D_out = 64, 1000, 100, 10
5 x, y = randn(N, D_in), randn(N, D_out)
6 w1, w2 = randn(D_in, H), randn(H, D_out)
7
8 for t in range(2000):
9     h = 1 / (1 + np.exp(-x.dot(w1)))
10    y_pred = h.dot(w2)
11    loss = np.square(y_pred - y).sum()
12    print(t, loss)
13
14    grad_y_pred = 2.0 * (y_pred - y)
15    grad_w2 = h.T.dot(grad_y_pred)
16    grad_h = grad_y_pred.dot(w2.T)
17    grad_w1 = x.T.dot(grad_h * h * (1 - h))
18
19    w1 -= 1e-4 * grad_w1
20    w2 -= 1e-4 * grad_w2
```

In Assignment 2: Writing a 2-layer net

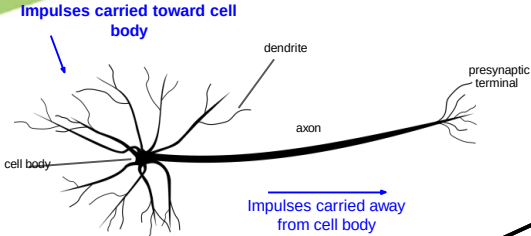
```
# receive W1,W2,b1,b2 (weights/biases), X (data)
# forward pass:
h1 = #... function of X,W1,b1
scores = #... function of h1,W2,b2
loss = #... (several lines of code to evaluate Softmax loss)
# backward pass:
dscores = #...
dh1,dW2,db2 = #...
dW1,db1 = #...
```



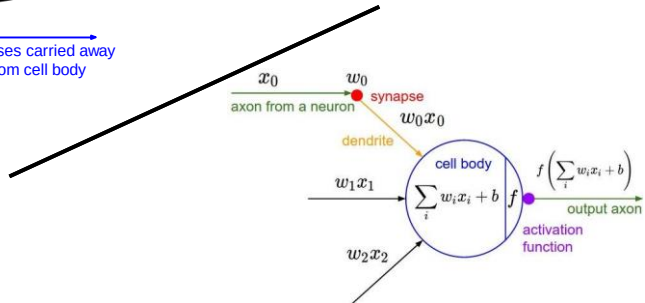
This image by Fotis Bobolas is
licensed under [CC-BY 2.0](#)



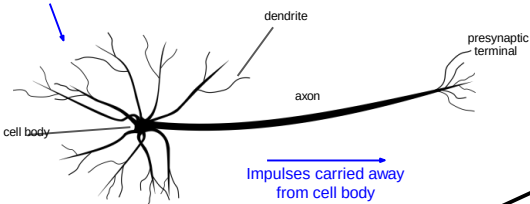
This image by Felipe Perucho
is licensed under [CC-BY 3.0](https://creativecommons.org/licenses/by/3.0/)



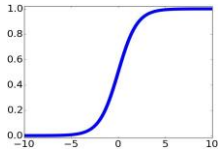
This image by Felipe Peruchio
is licensed under [CC-BY 3.0](https://creativecommons.org/licenses/by/3.0/)



Impulses carried toward cell body

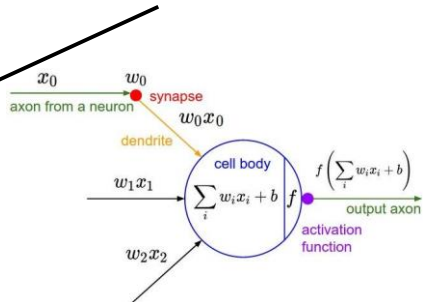


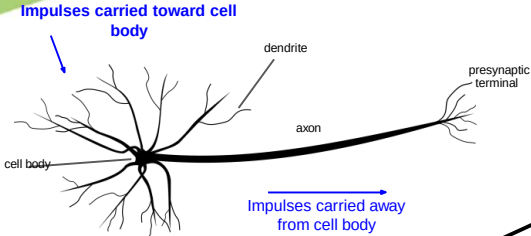
This image by Felipe Peruchio is licensed under [CC-BY 3.0](https://creativecommons.org/licenses/by/3.0/)



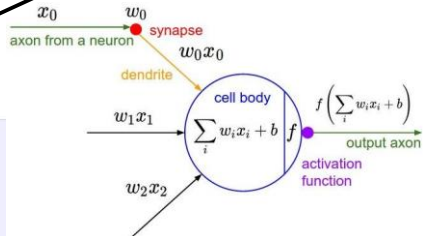
sigmoid activation function

$$\frac{1}{1 + e^{-x}}$$





This image by Felipe Peruchio
 is licensed under [CC-BY 3.0](https://creativecommons.org/licenses/by/3.0/)



```
class Neuron:
    # ...
    def neuron_tick(inputs):
        """ assume inputs and weights are 1-D numpy arrays and bias is a number """
        cell_body_sum = np.sum(inputs * self.weights) + self.bias
        firing_rate = 1.0 / (1.0 + math.exp(-cell_body_sum)) # sigmoid activation function
        return firing_rate
```

Be very careful with your brain analogies!

Biological Neurons:

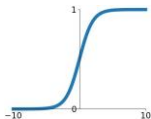
- Many different types
- Dendrites can perform complex non-linear computations
- Synapses are not a single weight but a complex non-linear dynamical system
- Rate code may not be adequate

[Dendritic Computation. London and Hausser]

Activation functions

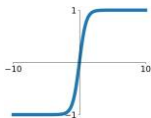
Sigmoid

$$\sigma(x) = \frac{1}{1+e^{-x}}$$



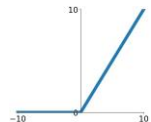
tanh

$$\tanh(x)$$



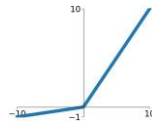
ReLU

$$\max(0, x)$$



Leaky ReLU

$$\max(0.1x, x)$$

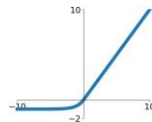


Maxout

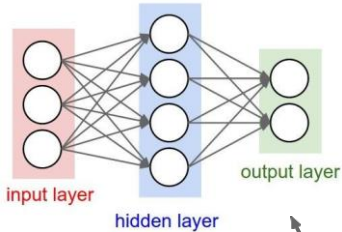
$$\max(w_1^T x + b_1, w_2^T x + b_2)$$

ELU

$$\begin{cases} x & x \geq 0 \\ \alpha(e^x - 1) & x < 0 \end{cases}$$

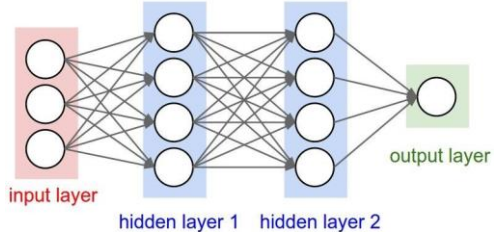


Neural networks: Architectures



“2-layer Neural Net”, or
“1-hidden-layer Neural Net”

“Fully-connected” layers



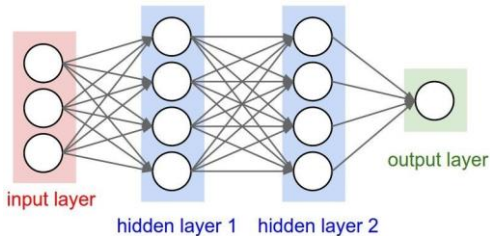
“3-layer Neural Net”, or
“2-hidden-layer Neural Net”

Example feed-forward computation of a neural network

```
class Neuron:
    # ...
    def neuron_tick(inputs):
        """ assume inputs and weights are 1-D numpy arrays and bias is a number """
        cell_body_sum = np.sum(inputs * self.weights) + self.bias
        firing_rate = 1.0 / (1.0 + math.exp(-cell_body_sum)) # sigmoid activation function
        return firing_rate
```

We can efficiently evaluate an entire layer of neurons.

Example feed-forward computation of a neural network



```
# forward-pass of a 3-layer neural network:
f = lambda x: 1.0/(1.0 + np.exp(-x)) # activation function (use sigmoid)
x = np.random.randn(3, 1) # random input vector of three numbers (3x1)
h1 = f(np.dot(W1, x) + b1) # calculate first hidden layer activations (4x1)
h2 = f(np.dot(W2, h1) + b2) # calculate second hidden layer activations (4x1)
out = np.dot(W3, h2) + b3 # output neuron (1x1)
```

Summary

- We arrange neurons into fully-connected layers
- The abstraction of a **layer** has the nice property that it allows us to use efficient vectorized code (e.g. matrix multiplies)
- Neural networks are not really *neural*
- Next time: Convolutional Neural Networks